

TECHNICAL DOCUMENTATION

Task Management System

End-to-End Technical Specification & Implementation Report



Submitted to

PT. Sompo Insurance Indonesia

Technical Assessment Deliverable

Spring Boot 3.x · Angular 16+ · Microsoft SQL Server

Prepared By	Syahril Arfian Almazril
Position	Candidate - Sompo Insurance Assessment
Contact	azril4974@gmail.com
Document Version	1.0.0 (Final Release)
Date Issued	11 June 2026
Status	Production-Ready
Classification	Confidential - Internal Assessment User

Daftar Isi

1. Document Control	6
1.1. Revision History	6
1.2. Document Purpose	6
1.3. Intended Audience	6
1.4. External References	7
1.5. Terminology & Glossary.....	7
2. Executive Summary	8
2.1. Key Highlights	8
2.2. Solution Snapshot.....	9
3. Project Overview	10
3.1. Background & Context.....	10
3.2. Project Objectives.....	10
3.3. Project Scope	10
3.3.1. In-Scope	10
3.3.2. Out-of-Scope.....	10
3.4. Assumptions & Constraints	11
4. Functional Requirements	12
4.1. Authentication & Authorization (FR-AUTH)	12
4.2. Task Management - CRUD (FR-TASK)	12
4.3. Validation & Search (FR-VS).....	13
4.4. Use Case Summary	13
5. Non-Functional Requirements	15
6. Technology Stack	16
6.1. Backend Stack.....	16
6.2. Frontend Stack.....	16
6.3. Database & Infrastructure	17
6.4. Development & Quality Tools	17
7. System Architecture	18
7.1. High-Level Architecture	18
7.2. Backend Layered Architecture	19
7.3. Frontend Component Architecture	19
7.4. Monorepo Project Structure	20
7.5. Request-Response Lifecycle	20
7.6. Architectural Principles.....	21

8. Database Design	22
8.1. Entity Relationship Diagram (ERD)	22
8.2. Table Specifications	22
8.2.1. Table: users.....	22
8.2.2. Table: tasks	23
8.3. Relationships & Constraints	23
8.4. Indexing Strategy	23
8.5. Data Initialization Strategy	24
9. User Flow & Use Cases	25
9.1. Global Navigation Map	25
9.2. UC-01: Login Flow	25
9.3. UC-02: Register Flow	26
9.4. UC-04: Create Task Flow (Admin only)	26
9.5. UC-08: Update Task Status Flow.....	26
9.6. UC-09: Delete Task Flow (Admin only)	27
9.7. UC-10: Logout / Token Expiry Flow	27
10. REST API Documentation	28
10.1. API Base URL & Conventions	28
10.2. Endpoint Summary	28
10.3. Authentication Endpoints.....	29
10.3.1. POST /api/auth/login	29
10.3.2. POST /api/auth/register	29
10.3.3. GET /api/auth/users	30
10.4. Task Management Endpoints	31
10.4.1. GET /api/tasks.....	31
10.4.2. GET /api/tasks/{id}.....	31
10.4.3. POST /api/tasks - Admin Only.....	32
10.4.4. PUT /api/tasks/{id} - Admin Only	32
10.4.5. PATCH /api/tasks/{id}/status.....	33
10.4.6. DELETE /api/tasks/{id} - Admin Only	33
10.5. Error Response Reference	34
11. Security Implementation	35
11.1. Authentication - JWT (JSON Web Token)	35
11.1.1. Token Structure	35
11.1.2. Issuance Flow.....	35
11.1.3. Validation Flow (per request).....	35

11.2. Password Hashing - BCrypt.....	35
11.3. Authorization - Role-Based Access Control (RBAC)	36
11.3.1. Layer 1 - Method-Level via @PreAuthorize.....	36
11.3.2. Layer 2 - Service-Level Ownership Validation	36
11.4. CORS Configuration	36
11.5. Stateless Session Policy	37
11.6. Security Best Practices Implemented	37
12. Backend Implementation	38
12.1. Application Entry Point	38
12.2. Security Configuration	38
12.3. JWT Components.....	39
12.3.1. JwtTokenProvider	39
12.3.2. JwtAuthenticationFilter	39
12.4. Domain Entities (Model Layer)	39
12.5. Repository Layer	40
12.6. Service Layer - Business Logic.....	41
12.6.1. AuthService - Login Implementation	41
12.6.2. TaskService - Filter Logic.....	41
12.7. Controller Layer	41
12.8. Global Exception Handling.....	42
12.9. Data Seeder	42
13. Frontend Implementation	44
13.1. Application Bootstrap.....	44
13.2. Routing & Guards	44
13.2.1. Guards.....	44
13.3. HTTP Interceptor - Otomatisasi JWT	44
13.4. Service Layer (HttpClient Wrappers)	45
13.4.1. AuthService.....	45
13.4.2. TaskService	45
13.5. Component Highlights	46
13.5.1. LoginComponent - Reactive Form	46
13.5.2. TaskListComponent - Adaptive Display	46
13.5.3. DashboardComponent - Aggregate Statistics.....	46
13.6. UI/UX Highlights	47
14. Environment Configuration	48
14.1. Backend - application.properties	48

14.2. Backend - Template. env.example	48
14.3. Frontend - environment.ts	49
14.4. .gitignore - Daftar Eksklusi Penting.....	49
14.5. Profil Konfigurasi per Environment	50
15. Database Schema & Seed Data	51
15.1. DDL - Tabel users	51
15.2. DDL - Tabel tasks.....	51
15.3. Seed Data - Pengguna Default.....	51
15.4. Seed Data - Contoh Tugas.....	52
16. Installation & Deployment Guide	53
16.1. Prasyarat Sistem	53
16.2. Setup Database SQL Server	53
16.3. Setup Backend	53
16.4. Setup Frontend.....	54
16.5. Build Produksi.....	54
16.5.1. Backend - Executable JAR	54
16.5.2. Frontend - Static Bundle	55
16.6. Catatan Deployment Produksi.....	55
17. Testing & Quality Assurance.....	56
17.1. Static Code Analysis - SonarCloud	56
17.1.1. Ringkasan Quality Gate.....	56
17.2. Manual Testing Checklist.....	56
17.2.1. Authentication & Authorization	56
17.2.2. Task Management	56
17.2.3. UI / UX.....	57
17.3. API Testing - Postman Collection.....	57
18. Default Credentials & Akses	58
18.1. Akun Bawaan	58
18.2. Peringatan Keamanan	58
19. Compliance Matrix - Pemenuhan Brief Teknis	59
20. Appendix - Tautan Resmi & Sumber Verifikasi	61
20.1. Struktur Penyerahan.....	61
20.2. Kontak.....	61
20.3. Pernyataan Penutup	61
21. Dokumentasi.....	62

1. Document Control

1.1. Revision History

Version	Date	Author	Description
0.3	08 Jun 2026	S. A. Almazril	Database schema definition; backend project scaffolding.
0.5	09 Jun 2026	S. A. Almazril	Backend implementation - JWT, RBAC, CRUD, validation, exception handler.
0.7	10 Jun 2026	S. A. Almazril	Frontend implementation - Angular Material UI, guards, interceptor.
0.9	10 Jun 2026	S. A. Almazril	Integration testing, Postman collection, SonarCloud analysis.
1.0	11 Jun 2026	S. A. Almazril	Final release - complete documentation; production-ready handover.

1.2. Document Purpose

Dokumen ini merupakan dokumentasi teknis end-to-end dari aplikasi Task Management System yang dikembangkan sebagai bagian dari Technical Assessment PT. Sampo Insurance Indonesia. Dokumen ini berfungsi sebagai referensi lengkap bagi tim teknis pereview (technical reviewer, user, dan technical lead) untuk memahami arsitektur, desain sistem, alur kerja, implementasi keamanan, serta tata cara instalasi dan deployment aplikasi.

Dokumentasi disusun mengikuti standar industri pengembangan perangkat lunak dengan tujuan agar aplikasi dapat dengan mudah dipahami, dipelihara, dan dilanjutkan pengembangannya oleh tim engineering Sampo Insurance, sekaligus sebagai Internal Assessment User untuk posisi Java Developer.

1.3. Intended Audience

- **Technical Reviewer / Hiring Engineer** - evaluasi kualitas teknis (clean code, security, error handling, scalability).
- **Backend Developer** - referensi struktur project, arsitektur layer, implementasi Spring Boot 3.x.
- **Frontend Developer** - referensi struktur Angular, state management, integrasi REST API & JWT interceptor.
- **Database Administrator (DBA)** - referensi skema relasional, constraint, dan seed data Microsoft SQL Server.
- **DevOps / SRE Engineer** - referensi environment variables, deployment pipeline, dan konfigurasi production.
- **QA Engineer / Tester** - referensi acceptance criteria, alur uji, dan endpoint Postman collection.

1.4. External References

Reference	Description	Location / URL
Source Code Repository	Monorepo (Backend + Frontend).	github.com/Arfazrll/Sampo-Insurance-Assessment
Postman API Collection	Dokumentasi endpoint interaktif lengkap dengan contoh request/response.	documenter.getpostman.com/view/39986471/2sBXwsK9br
SonarCloud Quality Report	Laporan analisis kualitas kode (security, maintainability, bug).	sonarcloud.io/summary/overall?id=Arfazrll_Sampo-Insurance-Assessment
Technical Assessment Brief	Dokumen kebutuhan resmi dari Sampo Insurance.	Internal - Sampo Insurance

1.5. Terminology & Glossary

Term	Definition
JWT	JSON Web Token - token bearer untuk otentikasi stateless yang ditransmisikan via header Authorization.
RBAC	Role-Based Access Control - model otorisasi berbasis peran (ROLE_ADMIN, ROLE_USER).
DTO	Data Transfer Object - objek pembawa data antar layer untuk memisahkan kontrak API dari entitas database.
BCrypt	Algoritma hashing password berbasis Blowfish dengan adaptive cost factor (default strength = 10).
CORS	Cross-Origin Resource Sharing - mekanisme browser untuk mengizinkan request lintas origin.
JPA	Java Persistence API - spesifikasi ORM standar yang diimplementasikan oleh Hibernate.
SPA	Single Page Application - aplikasi web yang dimuat sekali lalu navigasi dilakukan tanpa reload.
RESTful API	Application Programming Interface yang mengikuti prinsip Representational State Transfer.
Guard	Komponen Angular yang mengontrol akses ke rute (route activation).
Interceptor	Middleware HTTP Angular yang memodifikasi request/response secara global.

2. Executive Summary

Task Management System adalah aplikasi web berbasis arsitektur client-server yang dirancang untuk mengelola daftar tugas harian tim secara terstruktur dan aman. Aplikasi ini mengimplementasikan operasi CRUD penuh (Create, Read, Update, Delete) atas entitas tugas dengan sistem otentikasi berbasis token JSON Web Token (JWT) serta otorisasi berbasis peran (Role-Based Access Control / RBAC).

Aplikasi terdiri dari tiga komponen utama: Backend RESTful API yang dibangun di atas Spring Boot 3.3.5 dengan Java 17 sebagai runtime; Frontend Single-Page Application yang dibangun menggunakan Angular 16 dengan Angular Material sebagai komponen UI; dan Microsoft SQL Server sebagai penyimpanan data persisten yang diakses melalui Spring Data JPA dan driver JDBC resmi (mssql-jdbc).

Sistem mendukung dua peran pengguna utama, yaitu ROLE_ADMIN dengan hak akses penuh untuk mengelola seluruh tugas, dan ROLE_USER dengan hak akses terbatas untuk melihat serta memperbarui status tugas yang ditugaskan kepada dirinya sendiri. Keamanan diimplementasikan melalui enkripsi password dengan BCrypt, validasi token JWT di setiap request yang dilindungi, HTTP Interceptor pada sisi frontend yang menyisipkan token secara otomatis, serta Route Guards untuk mencegah akses tidak terautentikasi.

Aplikasi telah lulus pengujian kualitas kode menggunakan SonarCloud yang mencakup metrik security, reliability, dan maintainability. Seluruh kebutuhan fungsional dan teknis yang tercantum dalam Technical Assessment Brief telah diimplementasikan, diuji, dan didokumentasikan sesuai standar industri dan siap untuk dipromosikan ke environment production.

2.1. Key Highlights

- **Architecture:** Clean layered architecture mengikuti prinsip Separation of Concerns (Controller → Service → Repository → Model) dengan dependency injection via constructor.
- **Security:** JWT Bearer Token, BCrypt password hashing dengan cost factor 10, method-level @PreAuthorize, stateless session policy, dan CORS terkonfigurasi via environment variable.
- **Validation:** Bean Validation (Jakarta @Valid) di sisi backend, Angular Reactive Forms dengan dynamic error message di sisi frontend.
- **Error Handling:** Global Exception Handler terpusat dengan format ApiResponse yang konsisten untuk semua jenis error (validasi, autentikasi, otorisasi, business logic, server error).
- **Code Quality:** Lulus analisis SonarCloud dengan rating yang memenuhi quality gate standar industri.
- **Documentation:** Postman Collection, README, schema-seed.sql, environment example, dan dokumen teknis end-to-end ini.
- **Developer Experience:** Konfigurasi via .env file, automatic data seeding, environment.example.ts untuk frontend, hot-reload via Spring DevTools & Angular CLI.

2.2. Solution Snapshot

Aspect	Value
Application Name	Task Management System (TMS)
Architecture Pattern	Layered Architecture · RESTful API · Stateless Session
Backend Technology	Spring Boot 3.3.5 (Java 17)
Frontend Technology	Angular 16 (TypeScript) + Angular Material
Database	Microsoft SQL Server 2019+
Authentication	JWT Bearer Token (HS256, 24-hour expiry default)
Authorization	Role-Based Access Control (RBAC) - ROLE_ADMIN, ROLE_USER
Password Hashing	BCrypt (strength 10)
Build Tools	Maven (backend) · npm (frontend)
Lines of Code (approx.)	Backend ~1,800 LOC · Frontend ~2,400 LOC
API Endpoints	10 endpoints (3 Auth + 7 Tasks)
Code Quality Tool	SonarCloud

3. Project Overview

3.1. Background & Context

Aplikasi ini dirancang untuk mendukung skenario kerja kolaboratif di mana seorang Administrator dapat membuat, mendelegasikan, memantau, dan menghapus tugas, sementara setiap Anggota Tim (User) dapat memperbarui status tugas yang menjadi tanggung jawabnya. Setiap aksi tercatat dengan stempel waktu (timestamp) untuk audit dan transparansi.

3.2. Project Objectives

1. Membangun aplikasi web Task Management System yang memenuhi seluruh kebutuhan fungsional dan teknis dalam Technical Assessment Brief Sampo Insurance.
2. Mendemonstrasikan kompetensi teknis dalam pengembangan full-stack - mencakup Spring Boot 3.x sebagai backend, Angular 16+ sebagai frontend, dan Microsoft SQL Server sebagai database.
3. Mengimplementasikan praktik terbaik (best practices) industri di setiap layer - clean code, separation of concerns, security-by-design, dan error handling yang konsisten.
4. Menghasilkan codebase yang lulus quality gate SonarCloud serta dokumentasi end-to-end yang siap diserahkan untuk handover ke tim engineering.
5. Menyediakan deliverable yang production-ready: skrip seed database, file environment template, Postman collection, dan panduan deployment.

3.3. Project Scope

3.3.1. In-Scope

- Sistem autentikasi & otorisasi berbasis JWT dan RBAC (ROLE_ADMIN, ROLE_USER).
- Operasi CRUD penuh atas entitas tugas (Task) dengan relasi ke User.
- Fitur pencarian (search) berdasarkan judul tugas dan filter berdasarkan status.
- Validasi input di sisi backend (Bean Validation) dan frontend (Reactive Forms).
- Dashboard ringkasan statistik tugas untuk monitoring.
- Antarmuka yang responsif (mobile-friendly) menggunakan Angular Material.
- Global error handling dengan respons API yang konsisten.
- Seed data otomatis untuk akun admin & user serta sample tasks.
- Dokumentasi REST API via Postman Collection dan dokumen teknis ini.

3.3.2. Out-of-Scope

- Notifikasi email atau push notification (dapat ditambahkan sebagai future enhancement).
- Upload file/lampiran ke tugas (storage bucket integration).

- Kolaborasi real-time (WebSocket / Server-Sent Events).
- Auditing trail granular per field (di luar createdAt & updatedAt).
- Multi-tenancy (organization/workspace isolation).
- Integrasi pihak ketiga (SSO, OAuth provider, calendar sync).

3.4. Assumptions & Constraints

Category	Detail
Runtime Assumption	Server menyediakan JDK 17+, Node.js 16+, dan Microsoft SQL Server 2019+.
Network Assumption	Backend dan frontend dapat saling terhubung (default: localhost) dengan CORS terkonfigurasi.
Browser Support	Browser modern yang mendukung ES2020+ (Chrome 90+, Firefox 88+, Edge 90+, Safari 14+).
Time Zone	Server dan database menggunakan UTC sebagai default; konversi ditangani di client.
Language	Antarmuka pengguna menggunakan Bahasa Indonesia; kode dan komentar dalam Bahasa Inggris/Indonesia sesuai konteks.
Security Constraint	JWT secret minimal 256-bit (HS256). Tidak menyimpan secret di repository.
Database Constraint	Skema mengikuti relasi 1-to-many dari User ke Task.

4. Functional Requirements

Bagian ini menjelaskan setiap kebutuhan fungsional sebagaimana ditetapkan dalam Technical Assessment Brief, beserta keterangan status implementasi pada aplikasi Task Management System.

4.1. Authentication & Authorization (FR-AUTH)

ID	Requirement	Status
FR-AUTH-01	Pengguna dapat melakukan login menggunakan email & password.	Implemented
FR-AUTH-02	Pengguna dapat melakukan registrasi akun baru (default role: ROLE_USER).	Implemented
FR-AUTH-03	Sistem menerbitkan JWT token (Bearer) setelah autentikasi berhasil.	Implemented
FR-AUTH-04	Token JWT disisipkan otomatis di header Authorization pada setiap request.	Implemented
FR-AUTH-05	Pengguna dengan ROLE_ADMIN memiliki hak akses penuh atas semua tugas.	Implemented
FR-AUTH-06	Pengguna dengan ROLE_USER hanya dapat melihat & mengubah status tugas yang ditugaskan kepadanya.	Implemented
FR-AUTH-07	Semua endpoint API (kecuali /login dan /register) wajib dilindungi token JWT.	Implemented
FR-AUTH-08	Semua route frontend (kecuali login dan register) dilindungi Route Guard.	Implemented
FR-AUTH-09	Pengguna yang sudah login tidak dapat mengakses halaman login/register (GuestGuard).	Implemented
FR-AUTH-10	Token kedaluwarsa (HTTP 401) memicu auto-logout dan redirect ke halaman login.	Implemented
FR-AUTH-11	Sistem mencatat waktu login terakhir (last_login) pada saat autentikasi berhasil.	Implemented

4.2. Task Management - CRUD (FR-TASK)

ID	Requirement	Status
FR-TASK-01	ADMIN dapat membuat tugas baru dengan judul, deskripsi, status, dan assignee.	Implemented
FR-TASK-02	ADMIN & USER dapat melihat daftar tugas dengan view berbeda (semua vs milik sendiri).	Implemented
FR-TASK-03	ADMIN dapat mengubah keseluruhan data tugas (full update).	Implemented

ID	Requirement	Status
FR-TASK-04	ADMIN & USER (pemilik tugas) dapat mengubah status tugas (TODO / IN_PROGRESS / DONE).	Implemented
FR-TASK-05	ADMIN dapat menghapus tugas yang sudah ada.	Implemented
FR-TASK-06	Setiap tugas wajib memiliki relasi ke seorang User sebagai assignee.	Implemented
FR-TASK-07	Sistem otomatis menetapkan createdAt dan updatedAt timestamp via JPA lifecycle.	Implemented
FR-TASK-08	Tampilan daftar tugas tersedia dalam bentuk tabel (mat-table) yang responsif.	Implemented
FR-TASK-09	Dashboard menampilkan statistik agregat (Total, To Do, In Progress, Done).	Implemented
FR-TASK-10	User yang bukan assignee tidak dapat melihat/mengubah tugas (ForbiddenException).	Implemented

4.3. Validation & Search (FR-VS)

ID	Requirement	Status
FR-VS-01	Email pada login/register wajib valid (format @Email).	Implemented
FR-VS-02	Password minimal 8 karakter.	Implemented
FR-VS-03	Judul tugas tidak boleh kosong dan minimal 5 karakter.	Implemented
FR-VS-04	Field assignee tidak boleh kosong (@NotNull).	Implemented
FR-VS-05	Pencarian tugas berdasarkan kata kunci pada judul (case-insensitive).	Implemented
FR-VS-06	Filter tugas berdasarkan status (TODO / IN_PROGRESS / DONE).	Implemented
FR-VS-07	Kombinasi pencarian + filter status didukung secara bersamaan.	Implemented
FR-VS-08	Validasi gagal mengembalikan HTTP 400 dengan detail per-field.	Implemented
FR-VS-09	Reactive Form di frontend menampilkan pesan error dinamis per-field.	Implemented

4.4. Use Case Summary

Use Case	Actor	Description
UC-01: Login	Admin, User	Pengguna memasukkan email & password, sistem menerbitkan JWT dan mengarahkan ke dashboard.
UC-02: Register	Visitor	Visitor mendaftar akun baru dengan role default ROLE_USER, lalu otomatis login.

Use Case	Actor	Description
UC-03: View Dashboard	Admin, User	Pengguna melihat statistik agregat tugas dan ringkasan tugas terbaru.
UC-04: Create Task	Admin	Admin membuka modal Buat Tugas, mengisi field, dan menyimpan.
UC-05: View Task List	Admin, User	Pengguna melihat tabel tugas dengan pagination logis dan kemampuan filter/search.
UC-06: View Task Detail	Admin, User	Pengguna mengklik baris tugas untuk membuka modal Detail Tugas.
UC-07: Update Task	Admin	Admin membuka modal Edit dan memperbarui keseluruhan data tugas.
UC-08: Update Status	Admin, User (owner)	Pengguna mengubah status melalui dropdown inline pada baris tugas.
UC-09: Delete Task	Admin	Admin memilih tindakan Hapus pada baris tugas; modal konfirmasi muncul.
UC-10: Logout	Admin, User	Pengguna mengklik menu user dan memilih Keluar; token dihapus dan redirect ke /login.

5. Non-Functional Requirements

Selain kebutuhan fungsional, aplikasi Task Management System memenuhi sejumlah kebutuhan non-fungsional yang menjadi standar industri untuk aplikasi enterprise modern.

ID	Aspect	Specification
NFR-01	Performance	Response time endpoint REST < 300ms pada beban normal (development localhost).
NFR-02	Scalability	Stateless session policy memungkinkan horizontal scaling tanpa session replication.
NFR-03	Security	JWT (HS256, ≥256-bit secret), BCrypt strength 10, CORS whitelist, stateless session, no token in logs.
NFR-04	Reliability	Global exception handling memastikan tidak ada stack trace yang bocor ke client.
NFR-05	Maintainability	Clean layered architecture; SonarCloud quality gate Pass.
NFR-06	Portability	Konfigurasi via .env file; tidak ada credential hard-coded di source.
NFR-07	Usability	Responsive UI (mobile-first), Angular Material design system, error feedback dinamis.
NFR-08	Auditability	Setiap entitas memiliki created_at; entitas Task juga memiliki updated_at.
NFR-09	Compliance	Tidak menyimpan password plaintext; secret tidak masuk version control (.gitignore).
NFR-10	Internationalization	Antarmuka Bahasa Indonesia; teks pesan validasi dan error dalam Bahasa Indonesia.
NFR-11	Browser Compatibility	Chrome 90+, Firefox 88+, Edge 90+, Safari 14+.
NFR-12	Build Reproducibility	Maven Wrapper (mvnw) untuk backend; package-lock.json untuk frontend.

6. Technology Stack

Pemilihan teknologi didasarkan pada Technical Assessment Brief Sampo Insurance serta best practice industri untuk aplikasi enterprise modern. Setiap pilihan teknologi memiliki justifikasi yang jelas dari sisi kematangan ekosistem, dukungan komunitas, dan kemudahan deployment.

6.1. Backend Stack

Component	Technology	Version	Purpose
Language	Java	17 (LTS)	Runtime bahasa pemrograman backend.
Framework	Spring Boot	3.3.5	Application framework - autoconfiguration & opinionated defaults.
Web Layer	Spring Web (MVC)	3.3.5	RESTful API controllers.
Security	Spring Security	6.x	Authentication, authorization, security filter chain.
Persistence	Spring Data JPA + Hibernate	3.3.5 / 6.5	ORM untuk Microsoft SQL Server.
Validation	Jakarta Bean Validation (Hibernate Validator)	3.0	Validasi DTO request body.
JWT Library	JJWT (io.jsonwebtoken)	0.12.6	JWT issue, parse, dan validation.
DB Driver	Microsoft JDBC Driver	(managed)	Koneksi ke SQL Server.
Build Tool	Maven	3.8+	Dependency & build lifecycle management.
Boilerplate	Lombok	(managed)	Menghilangkan boilerplate (getter, setter, builder, constructor).
Testing	Spring Boot Test + JUnit 5	(managed)	Unit testing dan context loading.

6.2. Frontend Stack

Component	Technology	Version	Purpose
Framework	Angular	16	SPA framework dengan TypeScript.
Language	TypeScript	5.x	Strict-typed superset JavaScript.
UI Library	Angular Material + CDK	16	Komponen UI mengikuti Material Design 3.

Component	Technology	Version	Purpose
State Mgmt	RxJS + BehaviorSubject	7.x	Reactive state management.
HTTP Client	Angular HttpClient + Interceptor	16	Komunikasi REST API & otomatisasi JWT injection.
Forms	Angular Reactive Forms	16	Form handling dengan validasi dinamis.
Routing	Angular Router (lazy-eligible)	16	Client-side routing + Guards.
Styling	SCSS + Material Theming	-	Tema kustom dengan palette indigo & dark mode-ready.
Build Tool	Angular CLI (webpack)	16	Build & development server.

6.3. Database & Infrastructure

Component	Technology	Version / Notes
RDBMS	Microsoft SQL Server	2019 atau lebih baru
Schema Management	Hibernate DDL Auto (update) + manual schema-seed.sql	Hybrid (auto + scripted)
Encryption in Transit	TLS / TDS Encryption	encrypt=true di JDBC URL
Authentication Mode	SQL Server Authentication (username/password)	Configurable via .env
Seeding	Spring CommandLineRunner (DataSeeder.java)	Otomatis pada cold start

6.4. Development & Quality Tools

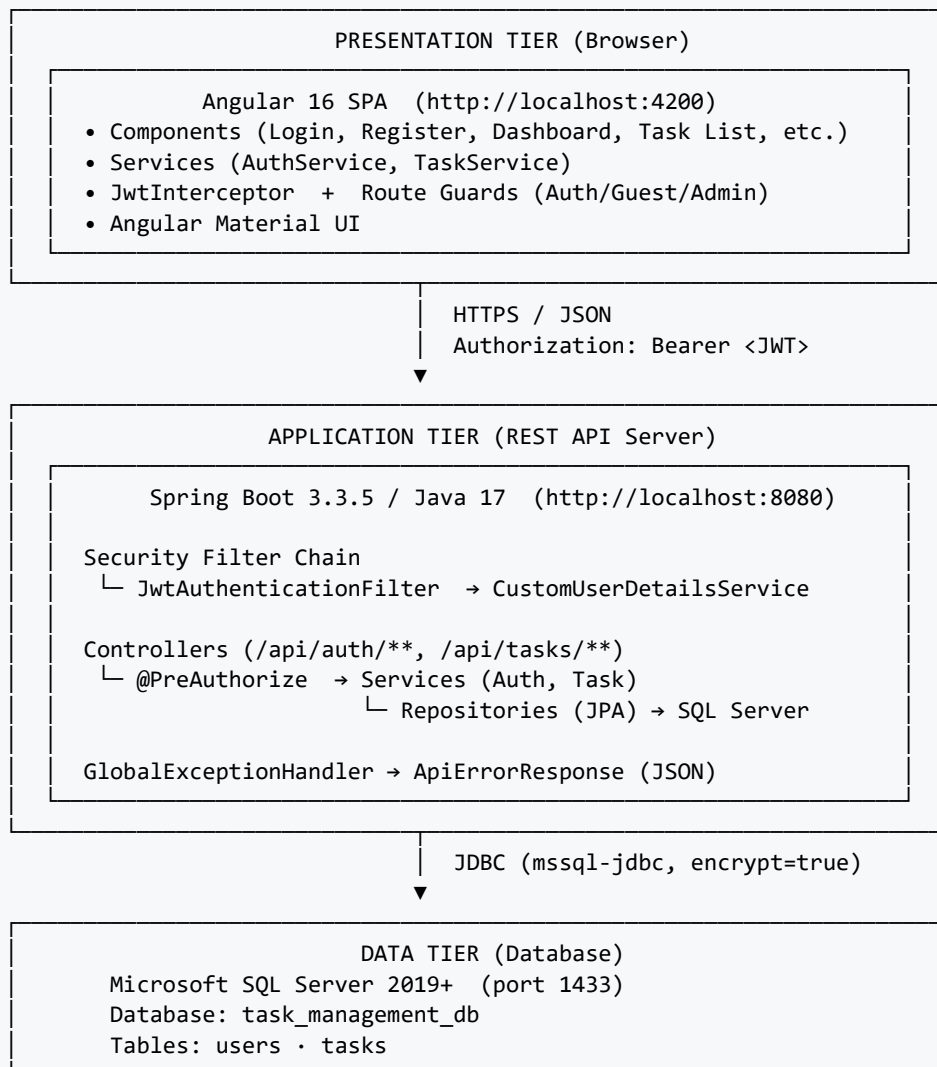
Tool	Purpose
Git + GitHub	Source control & remote repository.
Maven Wrapper (mvnw)	Memastikan konsistensi versi Maven antar mesin developer.
npm + package-lock.json	Dependency lock pada sisi frontend.
Postman	Manual API testing & dokumentasi interaktif.
SonarCloud	Static code analysis untuk security, bug, dan code smell.
Angular CLI	Code scaffolding, build, serve, dan testing utilities.
Spring Boot DevTools (opsional)	Hot reload untuk development.
IDE Recommendation	IntelliJ IDEA Ultimate (backend) + VS Code (frontend).

7. System Architecture

Bagian ini menjelaskan arsitektur aplikasi Task Management System dari sudut pandang high-level (deployment), komponen, serta struktur project secara fisik (folder layout).

7.1. High-Level Architecture

Aplikasi mengikuti pola arsitektur 3-tier yang terdiri dari Presentation Tier (Angular SPA), Application Tier (Spring Boot REST API), dan Data Tier (Microsoft SQL Server). Komunikasi antar tier dilakukan melalui protokol HTTPS dengan format payload JSON.



7.2. Backend Layered Architecture

Backend dibangun di atas pola Layered Architecture (juga dikenal sebagai N-tier architecture) yang memisahkan tanggung jawab secara horizontal. Setiap layer hanya berinteraksi dengan layer di bawahnya, mengikuti prinsip Dependency Inversion.

CONTROLLER LAYER (com.taskmanagement.controller) • AuthController → /api/auth/{login, register, users} • TaskController → /api/tasks/** • Responsibility: HTTP routing, request validation (@Valid), response shaping, @PreAuthorize role enforcement.
SERVICE LAYER (com.taskmanagement.service) • AuthService → login, register, getAllUsers • TaskService → CRUD + filtering + access validation • Responsibility: business logic, transactional boundaries, DTO ↔ Entity mapping, security context inspection.
REPOSITORY LAYER (com.taskmanagement.repository) • UserRepository extends JpaRepository<User, Long> • TaskRepository extends JpaRepository<Task, Long> • Responsibility: data access via JPA derived & @Query methods.
MODEL LAYER (com.taskmanagement.model) • User, Task (Entities) • Role, TaskStatus (Enums) • Responsibility: domain representation, JPA mapping, lifecycle hooks (@PrePersist, @PreUpdate).
CROSS-CUTTING • security/ → JwtTokenProvider, JwtAuthenticationFilter, CustomUserDetailsService, CustomUserDetails • config/ → SecurityConfig, DataSeeder • dto/ → request/* + response/* DTOs • exception/ → GlobalExceptionHandler + custom exceptions

7.3. Frontend Component Architecture

Frontend mengikuti pola modular Angular dengan pemisahan tugas antara core (singleton services & cross-cutting), features (domain-specific UI), dan shared (komponen reusable).

```

src/app/
├── core/
│   ├── guards/      → AuthGuard, GuestGuard, AdminGuard
│   ├── interceptors/ → JwtInterceptor (auto-injects Bearer token)
│   └── services/     → AuthService, TaskService (HttpClient wrappers)
├── features/
│   ├── auth/
│   │   ├── login/   → LoginComponent (reactive form)
│   │   ├── register/ → RegisterComponent
│   │   └── components/ → AuthIllustrationComponent (animated mascot)
│   ├── dashboard/   → DashboardComponent (stat cards + recent list)
│   └── tasks/

```

task-list/	→ Tabel tugas + search + filter
task-form/	→ Modal Create/Edit (Reactive Form)
task-detail-dialog	→ Modal Detail Tugas
layout/	→ LayoutComponent (sidenav + toolbar)
shared/	
components/	→ ConfirmDialogComponent
models/	→ TypeScript interfaces (Task, User, ApiError)
app.module.ts	→ Root NgModule (declarations + Material imports)

7.4. Monorepo Project Structure

Source code diatur dalam pola monorepo dengan dua folder utama: backend (Spring Boot) dan frontend (Angular). Pemisahan ini memudahkan independent deployment sekaligus tetap menjaga keterkaitan antar komponen.

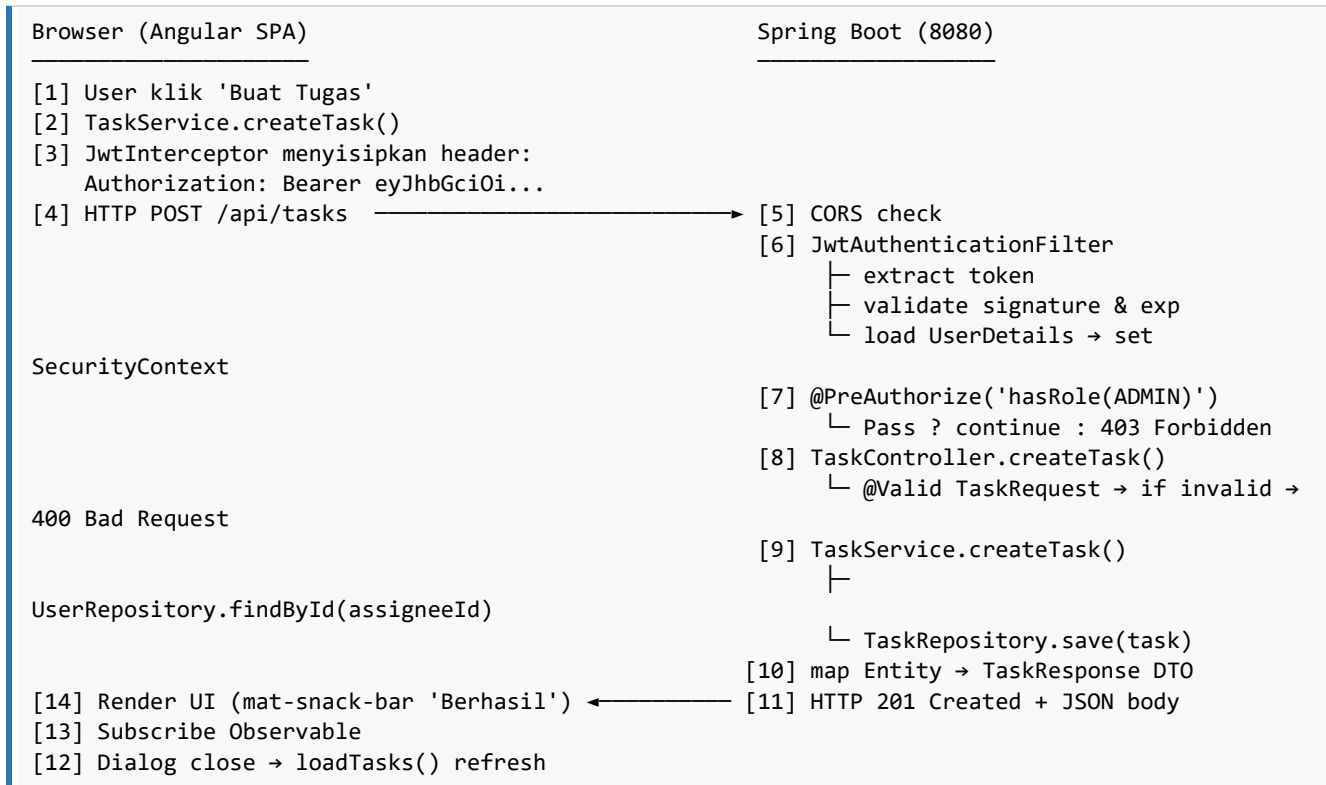
```

Sompo-Insurance-Assessment/
├── backend/
│   ├── src/main/java/com/taskmanagement/
│   │   ├── TaskManagementApplication.java # Spring Boot 3.x API
│   │   ├── config/ # @SpringBootApplication entry
│   │   ├── controller/ # SecurityConfig, DataSeeder
│   │   ├── dto/ # AuthController, TaskController
│   │   │   ├── request/
│   │   │   └── response/
│   │   ├── exception/ # LoginRequest, RegisterRequest, TaskRequest, ...
│   │   ├── model/ # AuthResponse, TaskResponse, ApiErrorResponse, ...
│   │   ├── repository/ # GlobalExceptionHandler + custom exceptions
│   │   ├── security/ # User, Task, Role, TaskStatus
│   │   └── service/ # UserRepository, TaskRepository
│   ├── src/main/resources/
│   │   ├── application.properties # JwtTokenProvider, JwtAuthenticationFilter, ...
│   │   └── schema-seed.sql # AuthService, TaskService
│   ├── src/test/java/com/taskmanagement/
│   ├── .env.example # Spring configuration (reads .env)
│   ├── mvnw / mvnw.cmd # Manual SQL bootstrap script
│   └── pom.xml # Spring Boot test context
├── frontend/
│   ├── src/
│   │   ├── app/ # Spring Boot test context
│   │   ├── assets/ # Environment template (commit-safe)
│   │   ├── environments/ # Maven Wrapper
│   │   ├── index.html # Maven dependencies
│   │   ├── main.ts # Angular 16 SPA
│   │   └── styles.scss # Komponen, services, guards, interceptors
│   ├── angular.json # environment.ts, environment.example.ts
│   ├── package.json # Global SCSS + Material theming
│   └── tsconfig.json
├── docs/
└── README.md # (Opsional) dokumentasi tambahan
# Panduan ringkas instalasi

```

7.5. Request-Response Lifecycle

Diagram berikut menggambarkan alur lengkap sebuah request dari browser hingga response, melewati seluruh komponen keamanan dan layer aplikasi.



7.6. Architectural Principles

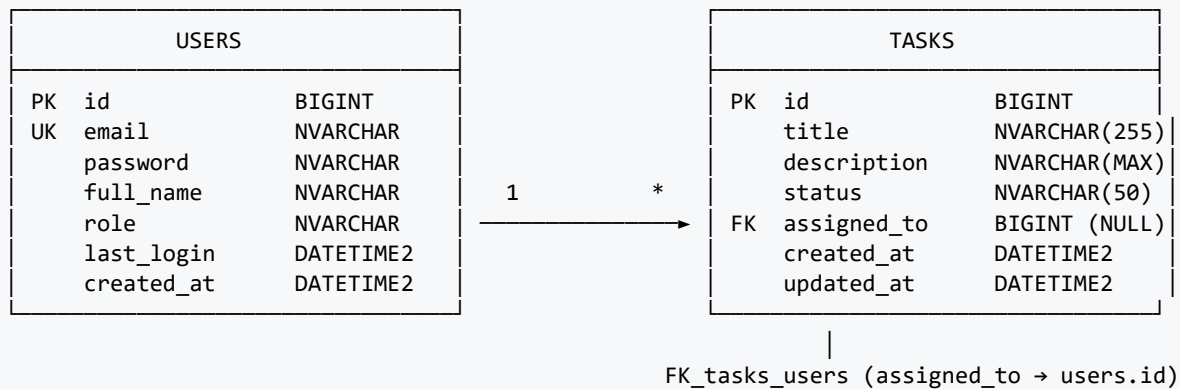
- **Separation of Concerns:** Setiap layer (Controller, Service, Repository, Model) memiliki tanggung jawab tunggal dan tidak boleh melompat layer.
- **Dependency Injection:** Semua dependency diinjeksi via constructor (didukung @RequiredArgsConstructor dari Lombok); tidak ada @Autowired field injection.
- **DTO Pattern:** Entitas tidak diekspos langsung ke klien. Layer DTO terpisah untuk request dan response.
- **Stateless Session:** Tidak ada HttpSession; otentikasi sepenuhnya bergantung pada JWT yang dikirim per request.
- **Defense in Depth:** Otorisasi diterapkan di dua lapis - @PreAuthorize pada controller dan validasi ownership pada service.
- **Configuration as Code & Environment:** Konfigurasi sensitif (DB password, JWT secret) tidak hard-coded; dikelola melalui .env.
- **Fail Fast:** Validasi input dijalankan sedini mungkin (di controller) sebelum menyentuh layer service/repository.

8. Database Design

Aplikasi menggunakan Microsoft SQL Server sebagai sistem manajemen basis data relasional. Skema didesain dengan dua entitas inti - users dan tasks - yang dihubungkan melalui relasi one-to-many (satu User dapat memiliki banyak Task).

8.1. Entity Relationship Diagram (ERD)

Diagram berikut menggambarkan struktur entitas dan relasi antar tabel pada database task_management_db.



Cardinality:

USER (1) — owns —> (0..N) TASK
 TASK (N) — assigned to — (1) USER

Enum domains (logical, enforced at application layer):

Role ∈ { ROLE_ADMIN, ROLE_USER }
 TaskStatus ∈ { TODO, IN_PROGRESS, DONE }

8.2. Table Specifications

8.2.1. Table: users

Tabel users menyimpan semua akun pengguna sistem, baik administrator maupun pengguna biasa.

Column	Type	Constraint	Description
id	BIGINT IDENTITY (1,1)	PRIMARY KEY	Surrogate key auto-increment.
email	NVARCHAR (255)	NOT NULL, UNIQUE	Identitas login pengguna; harus valid & unik.
password	NVARCHAR (255)	NOT NULL	Password terenkripsi BCrypt (cost 10). Tidak pernah disimpan dalam plaintext.
full_name	NVARCHAR (255)	NOT NULL	Nama lengkap pengguna.

Column	Type	Constraint	Description
role	NVARCHAR (50)	NOT NULL	Peran: ROLE_ADMIN atau ROLE_USER.
last_login	DATETIME2	NULL	Timestamp login terakhir (diupdate di AuthService).
created_at	DATETIME2	NOT NULL (default GETDATE ())	Timestamp pembuatan akun (di-set via @PrePersist).

8.2.2. Table: tasks

Tabel tasks menyimpan seluruh tugas yang dikelola sistem. Setiap tugas dikaitkan dengan satu User sebagai assignee.

Column	Type	Constraint	Description
id	BIGINT IDENTITY (1,1)	PRIMARY KEY	Surrogate key auto-increment.
title	NVARCHAR (255)	NOT NULL	Judul tugas (min. 5 karakter, divalidasi di layer service).
description	NVARCHAR(MAX)	NULL	Deskripsi tugas (opsional, long-text).
status	NVARCHAR (50)	NOT NULL, DEFAULT 'TODO'	Status tugas: TODO, IN_PROGRESS, DONE.
assigned_to	BIGINT	FOREIGN KEY → users(id), NULL	Referensi ke pengguna yang ditugaskan.
created_at	DATETIME2	NOT NULL (default GETDATE ())	Timestamp pembuatan (di-set via @PrePersist).
updated_at	DATETIME2	NOT NULL	Timestamp update terakhir (di-set via @PreUpdate).

8.3. Relationships & Constraints

Constraint Name	Type	Definition
PK_users	Primary Key	users(id)
UK_users_email	Unique Key	users(email)
PK_tasks	Primary Key	tasks(id)
FK_tasks_users	Foreign Key	tasks(assigned_to) → users(id)

8.4. Indexing Strategy

Microsoft SQL Server secara otomatis membuat clustered index pada Primary Key dan non-clustered unique index pada UNIQUE constraint. Untuk performa pencarian (search by title) dan filtering (status), index tambahan dapat dipertimbangkan pada deployment production dengan volume tinggi:

```
-- Disarankan untuk environment production dengan data > 100K tasks:
CREATE NONCLUSTERED INDEX IX_tasks_status ON tasks(status);
CREATE NONCLUSTERED INDEX IX_tasks_assigned_to ON tasks(assigned_to);
CREATE NONCLUSTERED INDEX IX_tasks_title ON tasks(title);

-- Composite index untuk kombinasi sering: filter status milik user tertentu
CREATE NONCLUSTERED INDEX IX_tasks_assigned_status
    ON tasks(assigned_to, status)
    INCLUDE (title, created_at);
```

8.5. Data Initialization Strategy

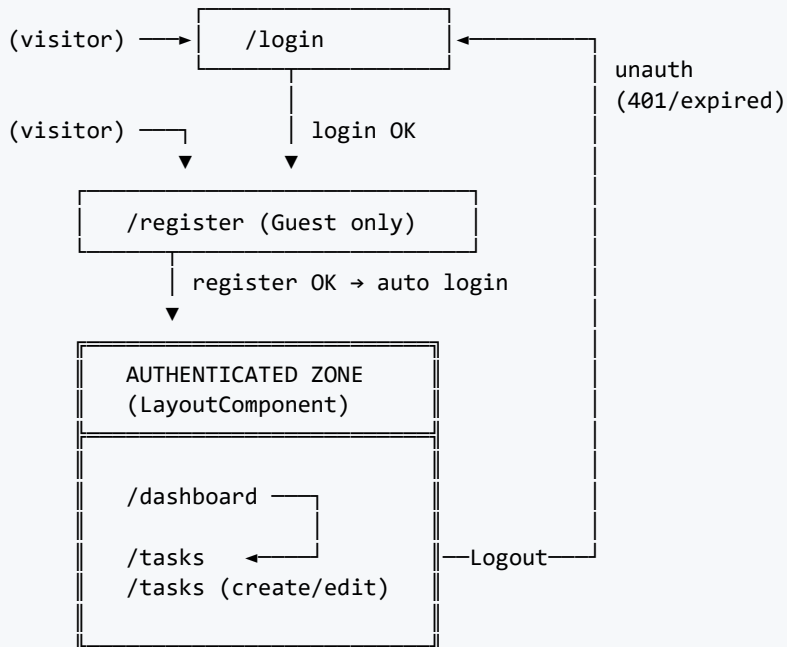
Sistem mendukung dua mekanisme inisialisasi skema dan seed data:

6. **Otomatis (default):** Spring Boot menjalankan Hibernate dengan ddl-auto=update yang membuat/memperbarui tabel sesuai entity. Setelah skema siap, CommandLineRunner DataSeeder.java mengeksekusi seed data hanya jika tabel users masih kosong (idempotent).
7. **Manual via SQL Script:** File backend/src/main/resources/schema-seed.sql menyediakan skrip CREATE TABLE dan INSERT yang idempotent (menggunakan IF NOT EXISTS) untuk skenario di mana DBA ingin mengelola schema secara eksplisit (production environment).

9. User Flow & Use Cases

Bagian ini menggambarkan alur interaksi pengguna dengan sistem dalam bentuk diagram tekstual yang mencakup happy path dan exception path untuk setiap skenario utama.

9.1. Global Navigation Map



9.2. UC-01: Login Flow

Actor: Admin or User

Pre: visitor di /login

1. User mengisi email & password pada form Reactive Forms.
2. Frontend memvalidasi: email valid (@Email), password ≥ 8 char.
3. Jika invalid → tampilkan pesan error inline; alur berhenti.
4. Jika valid → POST /api/auth/login dengan {email, password }.
5. Backend AuthenticationManager memvalidasi credential vs DB (BCrypt match).
6. Jika match → JwtTokenProvider generate token (HS256, 24h).
 - └ Update users.last_login = NOW ()
7. Backend respond 200 OK + {token, email, fullName, role }.
8. AuthService.handleAuthSuccess():
 - └ localStorage.setItem('auth_token', token)
 - └ localStorage.setItem('auth_user', JSON.stringify(authResponse))
 - └ currentUserSubject.next(authResponse) - propagate ke seluruh app.
9. Router.navigate(['/dashboard']).

Exception Paths:

- Credential salah → 401 Unauthorized + 'Email atau password salah'.
- Server down → HttpError di catchError → tampilkan error generic.
- Token format invalid → JwtAuthenticationFilter silent fail → 401 untuk request berikutnya.

9.3. UC-02: Register Flow

Actor: Visitor

Pre: visitor di /register

1. Visitor mengisi fullName, email, password.
2. Reactive Form memvalidasi field (required, email pattern, minLength).
3. POST /api/auth/register dengan {full Name, email, password }.
4. AuthService.register():
 - └ Cek existsByEmail → jika ada → throw BusinessException 'Email sudah terdaftar'.
 - └ passwordEncoder.encode(password) → BCrypt hash.
 - └ userRepository.save(user) dengan role = ROLE_USER.
 - └ Auto-login → generate JWT.
5. Respond 201 Created + {token, email, fullName, role }.
6. Frontend menyimpan token & redirect ke /dashboard.

Exception Paths:

- Email sudah ada → 400 Bad Request + message 'Email sudah terdaftar: <email>'.
- Validasi gagal → 400 + errors map: { full Name: '...', email: '...', password: '...' }.

9.4. UC-04: Create Task Flow (Admin only)

Actor: Admin

Pre: Admin sudah login

1. Admin klik tombol 'Buat Tugas' (visible hanya untuk Admin via *ngIf authService.isAdmin()).
2. TaskListComponent.openCreateDialog() membuka MatDialog dengan TaskFormComponent.
3. TaskFormComponent.ngOnInit() memuat daftar user via AuthService.getAllUsers() → GET /api/auth/users.
4. Admin mengisi title (≥5 char), description, status, assignedToId.
5. Reactive Form memvalidasi → tombol 'Buat' enabled jika valid.
6. Submit → TaskService.createTask() → POST /api/tasks (interceptor inject JWT).
7. Backend:
 - └ JwtAuthenticationFilter → autentikasi OK.
 - └ @PreAuthorize('hasRole (ADMIN)') → role check.
 - └ @Valid TaskRequest → validasi field.
 - └ TaskService.createTask():
 - └ userRepository.findById(assignedToId) atau ResourceNotFoundException.
 - └ taskRepository.save(task) - @PrePersist mengisi createdAt & updatedAt.
 - └ Respond 201 + TaskResponse.
8. Modal close + MatSnackBar 'Tugas berhasil dibuat'.
9. TaskListComponent.loadTasks() refresh daftar.

Exception Paths:

- non-admin user → 403 Forbidden dari @PreAuthorize.
- Assignee tidak ditemukan → 404 + 'User dengan ID X tidak ditemukan'.
- Validasi gagal → 400 + errors map per-field.

9.5. UC-08: Update Task Status Flow

Actor: Admin or User (owner of the task)

1. User membuka /tasks dan melihat tabel tugas yang relevan.
 - └ Jika Admin → semua tugas.
 - └ Jika User → hanya tugas yang assigned_to = current user.
2. User mengklik <mat-select> status pada baris tugas.

3. Pilih opsi baru (TODO / IN_PROGRESS / DONE).
4. TaskService.updateTaskStatus(id, {status}) → PATCH /api/tasks/{id}/status.
5. Backend:
 - └─ JwtAuthenticationFilter → autentikasi OK.
 - └─ TaskService.updateTaskStatus():
 - └─ findTaskOrThrow(id).
 - └─ validateAccess() → jika non-admin dan bukan owner → ForbiddenException.
 - └─ task.setStatus(TaskStatus.valueOf(request.status)).
 - └─ taskRepository.save(task) - @PreUpdate mengisi updatedAt = NOW().
 - └─ Respond 200 + TaskResponse.
6. MatSnackBar 'Status tugas berhasil diperbarui'.
7. loadTasks () refresh daftar (untuk konsistensi dengan filter aktif).

Exception Paths:

- User mencoba update tugas orang lain → 403 + 'Anda tidak memiliki akses ke tugas ini'.
- Task tidak ditemukan → 404 + 'Tugas dengan ID X tidak ditemukan'.
- Status tidak valid → 400 + IllegalArgumentException.

9.6. UC-09: Delete Task Flow (Admin only)

Actor: Admin

1. Admin klik icon 'delete' pada baris tugas.
2. MatDialog ConfirmDialogComponent terbuka - 'Yakin ingin menghapus tugas "X"?'.
3. Jika Admin pilih 'Batal' → dialog tertutup; tidak ada aksi.
4. Jika 'Ya, Hapus' → TaskService.deleteTask(id) → DELETE /api/tasks/{id}.
5. Backend @PreAuthorize ('hasRole (ADMIN)') → role check.
6. TaskService.deleteTask() → findTaskOrThrow → taskRepository.delete(task).
7. Respond 204 No Content.
8. MatSnackBar 'Tugas berhasil dihapus' + loadTasks().

9.7. UC-10: Logout / Token Expiry Flow

Trigger: User klik 'Keluar' di menu user ATAU HTTP 401 dari server

Manual logout:

1. User klik avatar → menu → 'Keluar'.
2. AuthService.logout():
 - └─ localStorage.removeItem('auth_token')
 - └─ localStorage.removeItem('auth_user')
 - └─ currentUserSubject.next(null)
 - └─ router.navigate(['/login']).

Token expired (auto detected):

1. Setiap request, JwtInterceptor menyisipkan Bearer token.
2. Backend JwtTokenProvider.validateToken() → expired → request lewati filter tanpa SecurityContext → endpoint protected → 401 Unauthorized.
3. JwtInterceptor.catchError → if (status===401 && ! url.includes('/auth/'))
→ authService.logout() → redirect /login.

10. REST API Documentation

Backend mengekspos REST API yang dikelompokkan menjadi dua resource utama: Authentication (/api/auth/**) dan Task Management (/api/tasks/**). Seluruh endpoint mengembalikan respons dalam format JSON dengan Content-Type application/json.

10.1. API Base URL & Conventions

Property	Value
Base URL (Development)	http://localhost:8080/api
Content-Type	application/json
Authentication Header	Authorization: Bearer <JWT_TOKEN>
Token Algorithm	HS256 (HMAC-SHA-256)
Token Expiration	86,400,000 ms (24 jam) - configurable via JWT_EXPIRATION
Error Format	ApiErrorResponse {status, message, timestamp, errors? }
Date Format	ISO 8601 (yyyy-MM-ddTHH:mm:ss)

10.2. Endpoint Summary

Method	Endpoint	Description	Access
POST	/api/auth/login	Otentikasi pengguna; menerbitkan JWT.	Public
POST	/api/auth/register	Registrasi user baru (role default: ROLE_USER).	Public
GET	/api/auth/users	Daftar semua pengguna sistem.	Authenticated
GET	/api/tasks	Daftar tugas (Admin: semua; User: milik sendiri).	Authenticated
GET	/api/tasks?search={q}&status={s}	Pencarian + filter status.	Authenticated
GET	/api/tasks/{id}	Detail tugas berdasarkan ID.	Authenticated*
POST	/api/tasks	Buat tugas baru.	ADMIN
PUT	/api/tasks/{id}	Update keseluruhan tugas.	ADMIN
PATCH	/api/tasks/{id}/status	Update status tugas saja.	Authenticated*
DELETE	/api/tasks/{id}	Hapus tugas.	ADMIN

(*) Akses dibatasi: User hanya dapat mengakses tugas yang ditugaskan kepadanya. Admin memiliki akses penuh.

10.3. Authentication Endpoints

10.3.1. POST /api/auth/login

Mengotentikasi pengguna dan menerbitkan JWT token.

Request Body

```
{
  "email": "admin@taskmanager.com",
  "password": "admin123"
}
```

Response - 200 OK

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOi...",
  "email": "admin@taskmanager.com",
  "fullName": "Administrator",
  "role": "ROLE_ADMIN"
}
```

Response - 401 Unauthorized (kredensial salah)

```
{
  "status": 401,
  "message": "Email atau password salah",
  "timestamp": "2026-06-11T10:23:14.567"
}
```

10.3.2. POST /api/auth/register

Mendaftarkan akun baru dengan role default ROLE_USER, dan secara otomatis melakukan auto-login.

Request Body

```
{
  "fullName": "Budi Santoso",
  "email": "budi@example.com",
  "password": "budi12345"
}
```

Response - 201 Created

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "email": "budi@example.com",
  "fullName": "Budi Santoso",
  "role": "ROLE_USER"
}
```

Response - 400 Bad Request (validation)

```
{
  "status": 400,
  "message": "Validasi gagal",
  "timestamp": "2026-06-11T10:25:00.000",
  "errors": {
    "email": "Format email tidak valid",
    "password": "Password minimal 8 karakter"
  }
}
```

10.3.3. GET /api/auth/users

Mengembalikan daftar seluruh pengguna terdaftar. Endpoint ini digunakan oleh form Create/Edit Task untuk memilih assignee.

Headers

Authorization: Bearer <JWT_TOKEN>

Response - 200 OK

```
[
  {
    "id": 1,
    "email": "admin@taskmanager.com",
    "fullName": "Administrator",
    "role": "ROLE_ADMIN",
    "lastLogin": "2026-06-11T10:23:14",
    "createdAt": "2026-06-07T08:00:00"
  },
  {
    "id": 2,
    "email": "user@taskmanager.com",
    "fullName": "John Doe",
    "role": "ROLE_USER",
    "lastLogin": null,
    "createdAt": "2026-06-07T08:00:00"
  }
]
```

10.4. Task Management Endpoints

10.4.1. GET /api/tasks

Mengembalikan daftar tugas yang dapat diakses oleh pengguna aktif. Admin mendapat seluruh tugas; User hanya mendapat tugas yang assigned kepadanya. Mendukung query parameter optional untuk search dan filter.

Query Parameters

Parameter	Type	Required	Description
search	String	No	Kata kunci pencarian pada field title (case-insensitive).
status	String	No	Filter berdasarkan status: TODO, IN_PROGRESS, atau DONE.

Example Request

```
GET /api/tasks?search=jwt&status=IN_PROGRESS
Authorization: Bearer <JWT>
```

Response - 200 OK

```
[
  {
    "id": 2,
    "title": "Implementasi autentikasi JWT",
    "description": "Membuat sistem login dengan Spring Security dan JWT token",
    "status": "IN_PROGRESS",
    "assignedTo": {
      "id": 2,
      "email": "user@taskmanager.com",
      "fullName": "John Doe"
    },
    "createdAt": "2026-06-09T13:30:00",
    "updatedAt": "2026-06-10T14:15:22"
  }
]
```

10.4.2. GET /api/tasks/{id}

Mengembalikan detail tugas berdasarkan ID. Mengembalikan 403 jika User mencoba akses tugas milik orang lain.

Response - 200 OK

```
{
  "id": 2,
  "title": "Implementasi autentikasi JWT",
  "description": "Membuat sistem login dengan Spring Security dan JWT token",
  "status": "IN_PROGRESS",
  "assignedTo": { "id": 2, "email": "user@taskmanager.com", "fullName": "John Doe" },
  "createdAt": "2026-06-09T13:30:00",
}
```

```
"updatedAt": "2026-06-10T14:15:22"
}
```

Response - 404 Not Found

```
{
  "status": 404,
  "message": "Tugas dengan ID 999 tidak ditemukan",
  "timestamp": "2026-06-11T10:30:00"
}
```

10.4.3. POST /api/tasks - Admin Only

Membuat tugas baru. Hanya dapat dilakukan oleh pengguna dengan role ROLE_ADMIN (dienforce oleh @PreAuthorize).

Request Body

```
{
  "title": "Review pull request #123",
  "description": "Cek implementasi fitur reset password",
  "status": "TODO",
  "assignedToId": 2
}
```

Response - 201 Created

```
{
  "id": 4,
  "title": "Review pull request #123",
  "description": "Cek implementasi fitur reset password",
  "status": "TODO",
  "assignedTo": { "id": 2, "email": "user@taskmanager.com", "fullName": "John Doe" },
  "createdAt": "2026-06-11T10:35:00",
  "updatedAt": "2026-06-11T10:35:00"
}
```

Response - 403 Forbidden (jika dipanggil oleh User non-Admin)

```
{
  "status": 403,
  "message": "Anda tidak memiliki izin untuk mengakses resource ini.",
  "timestamp": "2026-06-11T10:36:00"
}
```

10.4.4. PUT /api/tasks/{id} - Admin Only

Memperbarui keseluruhan data tugas. Hanya dapat dilakukan oleh ADMIN. Berbeda dari PATCH /status yang hanya mengubah satu field.

Request Body

```
{
  "title": "Review pull request #123 (revised)",
  "description": "Cek juga security audit",
}
```

```
{
  "status": "IN_PROGRESS",
  "assignedToId": 1
}
```

Response - 200 OK

(TaskResponse dengan data yang telah diperbarui, format sama dengan endpoint create)

10.4.5. PATCH /api/tasks/{id}/status

Mengubah hanya status tugas. Dapat dilakukan oleh Admin atau oleh User yang merupakan assignee tugas tersebut.

Request Body

```
{
  "status": "DONE"
}
```

Response - 200 OK

(TaskResponse dengan status terbaru, updatedAt = NOW())

Response - 403 Forbidden (User non-pemilik)

```
{
  "status": 403,
  "message": "Anda tidak memiliki akses ke tugas ini",
  "timestamp": "2026-06-11T10:40:00"
}
```

10.4.6. DELETE /api/tasks/{id} - Admin Only

Menghapus tugas. Hanya dapat dilakukan oleh ADMIN.

Response - 204 No Content

(empty body)

Response - 404 Not Found

```
{
  "status": 404,
  "message": "Tugas dengan ID 999 tidak ditemukan",
  "timestamp": "2026-06-11T10:42:00"
}
```

10.5. Error Response Reference

Seluruh error API ditangani oleh GlobalExceptionHandler dan dikembalikan dalam format ApiErrorResponse yang konsisten. Berikut adalah seluruh mapping status code yang digunakan:

HTTP Status	Trigger	Body
400 Bad Request	Bean Validation gagal (@Valid).	ApiErrorResponse dengan field errors map.
400 Bad Request	BusinessException (mis. email sudah terdaftar).	ApiErrorResponse dengan message custom.
401 Unauthorized	Tanpa Authorization header / token invalid.	ApiErrorResponse - entry point handler.
401 Unauthorized	Kredensial login salah.	BadCredentialsException → handler.
403 Forbidden	@PreAuthorize role gagal / ownership violation.	ApiErrorResponse - access denied handler.
404 Not Found	Resource tidak ada (task/user).	ResourceNotFoundException → 404.
500 Internal Server Error	Unhandled exception (fallback).	Generic 'Terjadi kesalahan pada server'.

11. Security Implementation

Aspek keamanan menjadi salah satu pilar utama dalam pengembangan aplikasi ini. Implementasi mengikuti prinsip Defense in Depth, di mana keamanan diterapkan di multiple layer (network, application, business logic, data).

11.1. Authentication - JWT (JSON Web Token)

Sistem menggunakan JWT sebagai mekanisme otentikasi stateless. Token diterbitkan saat login berhasil dan harus disertakan pada setiap request ke endpoint yang dilindungi.

11.1.1. Token Structure

```
Header (Base64URL):    { "alg": "HS256", "typ": "JWT" }
Payload (Claims):
  sub      : "admin@taskmanager.com"      // subject = email
  id       : 1                           // user ID
  fullName : "Administrator"              // display name
  roles    : "ROLE_ADMIN"                 // authorities
  iat      : 1717999800                    // issued at (Unix)
  exp      : 1718086200                    // expires (24h later)
Signature  : HMAC-SHA256( base64(header) + "." + base64(payload) , JWT_SECRET )
```

11.1.2. Issuance Flow

```
JwtTokenProvider.generateToken(Authentication):
├─ Cast principal → CustomUserDetails (id, email, fullName, authorities).
├─ Build claims: sub=email, id, fullName, roles=comma separated.
├─ Set iat = now, exp = now + JWT_EXPIRATION (default 86,400,000 ms).
└─ Sign with SecretKeySpec dari JWT_SECRET via HS256.
```

11.1.3. Validation Flow (per request)

```
JwtAuthenticationFilter.doFilterInternal():
1. Extract token dari header Authorization (strip prefix 'Bearer ').
2. Jika kosong → continue filter chain (anonymous).
3. jwtTokenProvider.validateToken(token):
   └─ Parse signed claims dengan secretKey.
      └─ Catch JwtException/IllegalArgumentException → return false.
4. Jika valid → loadUserByUsername(email) → set SecurityContext authentication.
5. filterChain.doFilter(request, response).
```

11.2. Password Hashing - BCrypt

Password disimpan dalam bentuk hash menggunakan algoritma BCrypt (cost factor 10 / default Spring Security). BCrypt menerapkan salt otomatis dan adaptive cost yang membuat brute-force attack lebih sulit seiring waktu.

```
// SecurityConfig.java
@Bean
```

```

public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(); // strength = 10 (default)
}

// AuthService.register()
User user = User.builder()
    .email(request.getEmail())
    .password(passwordEncoder.encode(request.getPassword())) // → "$2a$10$..."
    .role(Role.ROLE_USER)
    .build();

```

Validasi password saat login dilakukan otomatis oleh DaoAuthenticationProvider yang membandingkan plaintext input dengan stored hash menggunakan PasswordEncoder.matches().

11.3. Authorization - Role-Based Access Control (RBAC)

Sistem memiliki dua peran: ROLE_ADMIN dan ROLE_USER. Otorisasi diterapkan di dua lapis untuk memenuhi prinsip Defense in Depth.

11.3.1. Layer 1 - Method-Level via @PreAuthorize

```

// TaskController.java
@PostMapping
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<TaskResponse> createTask(@Valid @RequestBody TaskRequest request) {...}

@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<Void> deleteTask(@PathVariable Long id) {...}

// SecurityConfig.java
@EnableMethodSecurity // mengaktifkan @PreAuthorize

```

11.3.2. Layer 2 - Service-Level Ownership Validation

```

// TaskService.java
private void validateAccess(Task task) {
    CustomUserDetails currentUser = getCurrentUser();
    boolean isAdmin = currentUser.getAuthorities()
        .contains(new SimpleGrantedAuthority("ROLE_ADMIN"));

    if (!isAdmin && !task.getAssignedTo().getId().equals(currentUser.getId())) {
        throw new ForbiddenException("Anda tidak memiliki akses ke tugas ini");
    }
}

```

Validasi ownership ini memastikan bahwa User yang sudah login tetap tidak bisa mengakses tugas milik User lain, meskipun mereka melakukan request HTTP secara langsung di luar UI.

11.4. CORS Configuration

Cross-Origin Resource Sharing dikonfigurasi secara eksplisit untuk hanya menerima request dari origin yang terdaftar di environment variable CORS_ALLOWED_ORIGINS.

```
// SecurityConfig.java
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration cors = new CorsConfiguration();
    cors.setAllowedOrigins(List.of(allowedOrigins.split(",")));
    cors.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
    cors.setAllowedHeaders(List.of("*"));
    cors.setAllowCredentials(true);
    cors.setMaxAge(3600L);
    /* register for /** */
    return source;
}
```

11.5. Stateless Session Policy

```
// SecurityConfig.java
.sessionManagement(session -> session
    . sessionCreationPolicy(SessionCreationPolicy.STATELESS)
)
```

Tidak ada HttpSession yang dibuat. Setiap request bersifat independen dan harus membawa JWT sendiri. Pendekatan ini mendukung horizontal scaling tanpa perlu session replication dan menghilangkan vektor serangan session fixation.

11.6. Security Best Practices Implemented

Practice	Implementation
No plaintext password storage	Password di-hash dengan BCrypt sebelum disimpan.
No hard-coded secrets	JWT secret, DB credential dibaca dari .env (di-ignore oleh Git).
No token in logs	Spring logging tidak menampilkan Authorization header; show-sql=false.
CSRF protection	Dinonaktifkan eksplisit karena API stateless (csrf.disable()); JWT bertindak sebagai anti-CSRF.
Input validation	Bean Validation (@Valid) memblokir request sebelum mencapai service layer.
Output sanitization	DTO Response - password tidak pernah dikembalikan ke client.
Generic 500 error	GlobalExceptionHandler tidak mengekspos stack trace ke client.
Idempotent seed	DataSeeder dan schema-seed.sql menggunakan guard (IF NOT EXISTS / count check).
TLS in JDBC	encrypt=true di JDBC URL (TLS untuk koneksi ke SQL Server).
Method-level + service-level auth	Defense in Depth - @PreAuthorize + validateAccess().

12. Backend Implementation

Bagian ini menjelaskan implementasi backend secara mendalam, mulai dari entry point, konfigurasi keamanan, layer-by-layer hingga error handling. Seluruh kode mengikuti prinsip clean code, single responsibility, dan dependency inversion.

12.1. Application Entry Point

```
// TaskManagementApplication.java
@SpringBootApplication
public class TaskManagementApplication {
    public static void main (String[] args) {
        SpringApplication.run(TaskManagementApplication.class, args);
    }
}
```

@SpringBootApplication mengaktifkan auto-configuration, component scanning untuk package com.taskmanagement, dan bean definition exclusion sesuai dependency yang ada.

12.2. Security Configuration

Kelas SecurityConfig.java mengatur seluruh aspek keamanan: filter chain, password encoder, authentication manager, CORS, dan custom error handlers.

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthenticationFilter jwtAuthenticationFilter;
    @Value("${app.cors.allowed-origins}") private String allowedOrigins;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
                .anyRequest().authenticated())
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(...) // 401 JSON
                .accessDeniedHandler(...)) // 403 JSON
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

Custom AuthenticationEntryPoint dan AccessDeniedHandler memastikan bahwa error 401 dan 403 selalu dikembalikan dalam format JSON yang konsisten (ApiErrorResponse), bukan halaman HTML default Spring.

12.3. JWT Components

12.3.1. JwtTokenProvider

Tanggung jawab: generate, parse, dan validate JWT.

```
@Component
public class JwtTokenProvider {
    private final SecretKey secretKey;
    private final long jwtExpiration;

    public JwtTokenProvider(@Value("${app.jwt.secret}") String secret,
                           @Value("${app.jwt.expiration}") long expiration) {
        this.secretKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.jwtExpiration = expiration;
    }

    public String generateToken(Authentication auth) { /* build + sign claims */ }
    public String getEmailFromToken(String token)    { /* parse subject */ }
    public boolean validateToken(String token)        { /* try-catch parser */ }
}
```

12.3.2. JwtAuthenticationFilter

OncePerRequestFilter yang mengekstrak token dari header dan menetapkan SecurityContext.

```
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    private final JwtTokenProvider jwtTokenProvider;
    private final CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain
chain) {
        String token = extractTokenFromHeader(req);
        if (StringUtils.hasText(token) && jwtTokenProvider.validateToken(token)) {
            UserDetails ud = userDetailsService.loadUserByUsername(
                jwtTokenProvider.getEmailFromToken(token));
            var authToken = new UsernamePasswordAuthenticationToken(ud, null,
ud.getAuthorities());
            authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(req));
            SecurityContextHolder.getContext().setAuthentication(authToken);
        }
        chain.doFilter(req, res);
    }
}
```

12.4. Domain Entities (Model Layer)

```

@Entity @Table(name = "users")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable=false, unique=true) private String email;
    @Column(nullable=false) private String password;
    @Column(nullable=false) private String fullName;
    @Enumerated(EnumType.STRING) @Column(nullable=false) private Role role;
    @Column(name="last_login") private LocalDateTime lastLogin;
    @Column(name="created_at", updatable=false) private LocalDateTime createdAt;

    @PrePersist protected void onCreate() { this.createdAt = LocalDateTime.now(); }
}

@Entity @Table(name = "tasks")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class Task {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable=false) private String title;
    @Column(columnDefinition="NVARCHAR(MAX)") private String description;
    @Enumerated(EnumType.STRING) @Column(nullable=false) private TaskStatus status;
    @ManyToOne(fetch=FetchType.LAZY)
    @JoinColumn(name="assigned_to", referencedColumnName="id")
    private User assignedTo;
    @Column(name="created_at", updatable=false) private LocalDateTime createdAt;
    @Column(name="updated_at") private LocalDateTime updatedAt;

    @PrePersist protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
        if (this.status == null) this.status = TaskStatus.TODO;
    }
    @PreUpdate protected void onUpdate() { this.updatedAt = LocalDateTime.now(); }
}

```

12.5. Repository Layer

Repository menggunakan Spring Data JPA. Method derivation untuk query sederhana, @Query JPQL untuk query yang lebih kompleks (case-insensitive search & multi-criteria filtering).

```

@Repository
public interface TaskRepository extends JpaRepository<Task, Long> {
    List<Task> findByAssignedToId(Long userId);
    List<Task> findByStatus(TaskStatus status);
    List<Task> findByAssignedToIdAndStatus(Long userId, TaskStatus status);

    @Query("SELECT t FROM Task t WHERE LOWER(t.title) LIKE LOWER(CONCAT('%', :title, '%'))")
    List<Task> searchByTitle(@Param("title") String title);

    @Query("SELECT t FROM Task t WHERE LOWER(t.title) LIKE LOWER(CONCAT('%', :title, '%')) " +
        "AND t.status = :status")
    List<Task> searchByTitleAndStatus(@Param("title") String title, @Param("status") TaskStatus

```

```
status);
    /* additional combination queries... */
}
```

12.6. Service Layer - Business Logic

Service layer mengandung seluruh business logic, melakukan mapping Entity ↔ DTO, dan menjadi transactional boundary.

12.6.1. AuthService - Login Implementation

```
@Transactional
public AuthResponse login(LoginRequest request) {
    Authentication auth = authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(request.getEmail(), request.getPassword()));

    userRepository.findByEmail(request.getEmail()).ifPresent(user -> {
        user.setLastLogin(LocalDateTime.now());
        userRepository.save(user);
    });

    String token = jwtTokenProvider.generateToken(auth);
    User user = userRepository.findByEmail(request.getEmail()).orElseThrow();
    return AuthResponse.builder().token(token).email(user.getEmail())
        .fullName(user.getFullName()).role(user.getRole().name()).build();
}
```

12.6.2. TaskService - Filter Logic

Method filterTasks mengkombinasikan berbagai parameter input (search, status, userId) dan memilih query repository yang paling tepat - pendekatan ini menghindari N+1 query dan tetap eksplisit.

```
private List<Task> filterTasks(String search, TaskStatus status, Long userId) {
    boolean hasSearch = search != null && !search.trim().isEmpty();
    boolean hasStatus = status != null;
    boolean hasUserId = userId != null;

    if (hasSearch && hasStatus && hasUserId)
        return taskRepository.searchByTitleAndStatusAndUserId(search, status, userId);
    if (hasSearch && hasStatus) return taskRepository.searchByTitleAndStatus(search, status);
    if (hasSearch && hasUserId) return taskRepository.searchByTitleAndUserId(search, userId);
    if (hasStatus && hasUserId) return taskRepository.findByAssignedToIdAndStatus(userId,
status);
    if (hasSearch) return taskRepository.searchByTitle(search);
    if (hasStatus) return taskRepository.findByStatus(status);
    if (hasUserId) return taskRepository.findByAssignedToId(userId);
    return taskRepository.findAll();
}
```

12.7. Controller Layer

Controller hanya mendelegasikan ke service. Tanggung jawabnya adalah HTTP routing, validasi format request, dan menetapkan response status.

```
@RestController
@RequestMapping("/api/tasks")
@RequiredArgsConstructor
public class TaskController {
    private final TaskService taskService;

    @GetMapping
    public ResponseEntity<List<TaskResponse>> getAllTasks(
        @RequestParam(required=false) String search,
        @RequestParam(required=false) String status) {
        return ResponseEntity.ok(taskService.getAllTasks(search, status));
    }

    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<TaskResponse> createTask(@Valid @RequestBody TaskRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(taskService.createTask(request));
    }
    /* ... PUT, PATCH, DELETE ... */
}
```

12.8. Global Exception Handling

Seluruh exception ditangkap secara terpusat oleh `@RestControllerAdvice`, lalu dipetakan ke `ApiErrorResponse` dengan HTTP status yang sesuai.

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiErrorResponse> handleValidationErrors(...) { /* 400 + errors map */ }

    @ExceptionHandler(BadCredentialsException.class)
    public ResponseEntity<ApiErrorResponse> handleBadCredentials(...) { /* 401 */ }

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiErrorResponse> handleNotFound(...) { /* 404 */ }

    @ExceptionHandler(BusinessException.class)
    public ResponseEntity<ApiErrorResponse> handleBusinessException(...) { /* 400 */ }

    @ExceptionHandler(ForbiddenException.class)
    public ResponseEntity<ApiErrorResponse> handleForbidden(...) { /* 403 */ }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiErrorResponse> handleGenericException(...) { /* 500 generic */ }
}
```

12.9. Data Seeder

Pada cold start (saat tabel users masih kosong), DataSeeder otomatis membuat akun admin, user, dan beberapa sample task untuk testing.

```
@Component @RequiredArgsConstructor
public class DataSeeder implements CommandLineRunner {
```

```
private final UserRepository userRepository;
private final TaskRepository taskRepository;
private final PasswordEncoder passwordEncoder;

@Override
public void run(String... args) {
    if (userRepository.count() > 0) return; // idempotent guard

    User admin = userRepository.save(User.builder()
        .fullName("Administrator")
        .email("admin@taskmanager.com")
        .password(passwordEncoder.encode("admin123"))
        .role(Role.ROLE_ADMIN).build());

    User user = userRepository.save(User.builder()
        .fullName("John Doe")
        .email("user@taskmanager.com")
        .password(passwordEncoder.encode("user1234"))
        .role(Role.ROLE_USER).build());

    /* save 3 sample tasks */
}
```

13. Frontend Implementation

Frontend dibangun menggunakan Angular 16 dengan TypeScript dan Angular Material sebagai komponen UI library. Bagian ini menjabarkan implementasi modular dari setiap aspek frontend.

13.1. Application Bootstrap

```
// main.ts
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';

platformBrowserDynamic().bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

13.2. Routing & Guards

Aplikasi memiliki routing eager-loaded dengan dua mode utama: route publik (login/register) dan route terproteksi (di bawah LayoutComponent). Tiga jenis Guard memastikan akses sesuai status autentikasi.

```
// app-routing.module.ts
const routes: Routes = [
  { path: 'login', component: LoginComponent, canActivate: [GuestGuard] },
  { path: 'register', component: RegisterComponent, canActivate: [GuestGuard] },
  {
    path: '',
    component: LayoutComponent,
    canActivate: [AuthGuard],
    children: [
      { path: 'dashboard', component: DashboardComponent },
      { path: 'tasks', component: TaskListComponent },
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
    ]
  },
  { path: '**', redirectTo: 'dashboard' }
];
```

13.2.1. Guards

Guard	Behavior
AuthGuard	Mengizinkan akses jika user sudah login (token valid). Jika tidak → redirect /login.
GuestGuard	Sebaliknya - jika sudah login, blok akses ke /login & /register; redirect /dashboard.
AdminGuard	Memvalidasi role ROLE_ADMIN. (Disiapkan untuk halaman khusus admin di masa depan.)

13.3. HTTP Interceptor - Otomatisasi JWT

JwtInterceptor adalah middleware HTTP global yang menyisipkan token Bearer ke setiap request keluar dan menangani auto-logout pada response 401.

```

@Injectable()
export class JwtInterceptor implements HttpInterceptor {
  constructor(private authService: AuthService) {}

  intercept(req: HttpRequest<unknown>, next: HttpHandler) {
    const token = this.authService.getToken();
    if (token && this.authService.isLoggedIn()) {
      req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
    }
    return next.handle(req).pipe(
      catchError((error: HttpErrorResponse) => {
        if (error.status === 401 && !req.url.includes('/auth/')) {
          this.authService.logout();
        }
        return throwError(() => error);
      })
    );
  }
}

```

Interceptor didaftarkan di AppModule dengan multi=true untuk mendukung composability dengan interceptor lain di masa depan.

13.4. Service Layer (HttpClient Wrappers)

13.4.1. AuthService

AuthService bertanggung jawab atas seluruh interaksi terkait otentikasi: login, register, logout, manajemen token di localStorage, dan distribusi state via BehaviorSubject.

```

@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly TOKEN_KEY = 'auth_token';
  private readonly USER_KEY = 'auth_user';
  private currentUserSubject = new BehaviorSubject<AuthResponse|null>(this.getStoredUser());
  currentUser$ = this.currentUserSubject.asObservable();

  login(req: LoginRequest) { return this.http.post<AuthResponse>(`${API_URL}/login`, req)
    .pipe(tap(r => this.handleAuthSuccess(r))); }

  register(req: RegisterRequest) { return this.http.post<AuthResponse>(`${API_URL}/register`,
    req)
    .pipe(tap(r => this.handleAuthSuccess(r))); }

  logout() { localStorage.removeItem(TOKEN_KEY);
    localStorage.removeItem(USER_KEY);
    this.currentUserSubject.next(null);
    this.router.navigate(['/login']); }

  isLoggedIn(): boolean { /* decode JWT exp claim, compare dengan Date.now() */ }
  isAdmin(): boolean { return this.currentUserSubject.value?.role === 'ROLE_ADMIN'; }
}

```

13.4.2. TaskService

```

@Injectables({ providedIn: 'root' })
export class TaskService {
  getAllTasks(search?: string, status?: string) {
    let params = new HttpParams();
    if (search?.trim()) params = params.set('search', search.trim());
    if (status?.trim()) params = params.set('status', status.trim());
    return this.http.get<TaskResponse[]>(API_URL, { params });
  }
  getTaskById(id) { return this.http.get<TaskResponse>(`${API_URL}/${id}`); }
  createTask(req) { return this.http.post<TaskResponse>(API_URL, req); }
  updateTask(id, req) { return this.http.put<TaskResponse>(`${API_URL}/${id}`, req); }
  updateTaskStatus(id, r) { return this.http.patch<TaskResponse>(`${API_URL}/${id}/status`, r); }
  deleteTask(id) { return this.http.delete<void>(`${API_URL}/${id}`); }
}

```

13.5. Component Highlights

13.5.1. LoginComponent - Reactive Form

```

this.loginForm = this.fb.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', [Validators.required, Validators.minLength(8)]]
});

onSubmit() {
  if (this.loginForm.invalid) { this.loginForm.markAllAsTouched(); return; }
  this.authService.login(this.loginForm.value).subscribe({
    next: () => this.router.navigate(['/dashboard']),
    error: (e) => this.errorMessage = e.error?.message || 'Terjadi kesalahan'
  });
}

```

13.5.2. TaskListComponent - Adaptive Display

Komponen ini menampilkan kolom yang berbeda untuk Admin vs User berkat getter displayedColumns:

```

get displayedColumns(): string[] {
  return this.authService.isAdmin()
    ? ['title', 'status', 'assignedTo', 'createdAt', 'actions']
    : ['title', 'status', 'assignedTo', 'createdAt'];
}

```

13.5.3. DashboardComponent - Aggregate Statistics

```

loadDashboardData() {
  this.taskService.getAllTasks().subscribe({
    next: (tasks) => {
      this.tasks = tasks;
      this.totalTasks = tasks.length;
      this.todoCount = tasks.filter(t => t.status === 'TODO').length;
      this.inProgressCount = tasks.filter(t => t.status === 'IN_PROGRESS').length;
      this.doneCount = tasks.filter(t => t.status === 'DONE').length;
    }
  });
}

```

```
}  
getCompletionPercentage(): number {  
  return this.totalTasks === 0 ? 0 : Math.round((this.doneCount / this.totalTasks) * 100);  
}
```

13.6. UI/UX Highlights

- **Animated Login Illustration:** Empat karakter (purple, black, orange, yellow) yang mata-nya mengikuti pointer mouse - terinspirasi referensi UX modern. Diimplementasikan via `@HostListener('window:mousemove')` dengan kalkulasi posisi pupil per-karakter.
- **Glassmorphism Auth Card:** Backdrop-filter blur(20px), translucent background, dan animated gradient untuk mobile fallback.
- **Responsive Layout:** MatSidenav mode 'side' di desktop, 'over' di mobile via BreakpointObserver.
- **Color-Coded Status Badge:** Warna konsisten dengan stat card dashboard - TODO (amber), IN_PROGRESS (sky), DONE (emerald).
- **Modal Dialogs:** Material Dialog dengan panelClass 'modern-dialog' (border-radius 20px) untuk form create/edit, detail tugas, dan confirm hapus.
- **Snack-Bar Feedback:** Notifikasi sukses (hijau) dan error (merah) dengan posisi end-top, durasi 3000ms.
- **Optimistic UI Pattern:** Setelah operasi sukses, daftar diloat ulang untuk menjamin sinkronisasi dengan filter aktif.

14. Environment Configuration

Aplikasi dirancang dengan separation of concerns yang ketat antara source code dan secret/credential. Seluruh nilai sensitif diekstrak menjadi environment variable, sementara application.properties hanya menyimpan referensi placeholder beserta default yang aman untuk lingkungan pengembangan lokal.

14.1. Backend - application.properties

File berikut menjadi single source of truth bagi seluruh konfigurasi Spring Boot. Setiap properti dapat di-override melalui environment variable atau file .env (di-load via spring.config.import). Mekanisme ini menjamin portabilitas antar environment (development / staging / production) tanpa perlu rebuild artifact.

```
spring.application.name=task-management-system
spring.config.import=optional:file:.env[.properties]

# Server Configuration
server.port=8080

# Database Configuration (SQL Server)
spring.datasource.url=${DB_URL:jdbc:sqlserver://localhost:1433;databaseName=task_management_db;encrypt=true;trustServerCertificate=true}
spring.datasource.username=${DB_USERNAME:sa}
spring.datasource.password=${DB_PASSWORD:YourPassword123!}
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver

# JPA / Hibernate Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.SQLServerDialect
spring.jpa.properties.hibernate.format_sql=true

# JWT Configuration
jwt.secret=${JWT_SECRET:TaskManagementSecretKey2024SompoInsuranceAssessmentJWTSecretKeyMinimal256 Bits}
jwt.expiration=${JWT_EXPIRATION:86400000}

# CORS Configuration
cors.allowed-origins=${CORS_ALLOWED_ORIGINS:http://localhost:4200}
```

Pola `${ENV_VAR:default}` memberikan dua keuntungan: pertama, environment variable selalu memenangkan default sehingga kredensial produksi tidak pernah perlu dikomit ke repository; kedua, default yang disediakan memungkinkan klon repository langsung dapat dijalankan untuk evaluasi tanpa konfigurasi tambahan.

14.2. Backend - Template. env.example

Repository menyertakan .env.example sebagai template kontrak environment variable. File aktual. env tidak dikomit (terdaftar di .gitignore) dan harus dibuat secara manual oleh setiap operator.

```
# Database
DB_URL=jdbc:sqlserver://localhost:1433;databaseName=task_management_db;encrypt=true;trustServerCertificate=true
DB_USERNAME=sa
DB_PASSWORD=YourPassword123!

# JWT
JWT_SECRET=TaskManagementSecretKey2024SompoInsuranceAssessmentJWTSecretKeyMinimal256Bits
JWT_EXPIRATION=86400000

# CORS
CORS_ALLOWED_ORIGINS=http://localhost:4200
```

14.3. Frontend - environment.ts

Angular menggunakan environment files untuk membedakan konfigurasi build-time. File environment.ts dipakai pada ng serve / npm start, sedangkan environment.prod.ts diaktifkan otomatis melalui fileReplacements pada konfigurasi produksi (angular.json).

```
// src/environments/environment.ts
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8080/api'
};

// src/environments/environment.prod.ts
export const environment = {
  production: true,
  apiUrl: 'https://api.yourdomain.com/api'
};
```

14.4. .gitignore - Daftar Eksklusi Penting

```
# Backend
backend/.env
backend/.env.local
backend/target/
backend/*.log

# Frontend
frontend/node_modules/
frontend/dist/
frontend/.angular/
frontend/src/environments/environment.local.ts

# IDE
.idea/
.vscode/
*.iml
```

14.5. Profil Konfigurasi per Environment

Properti	Development	Staging	Production
spring.jpa.hibernate.ddl-auto	update	validate	none
spring.jpa.show-sql	false	false	false
jwt.expiration	86400000 (24h)	43200000 (12h)	3600000 (1h) - rekomendasi
cors.allowed-origins	http://localhost:4200	https://staging.app	https://app.yourdomain.com
server.port	8080	8080	8080 (di belakang reverse proxy)
trustServerCertificate	true	false	false (wajib CA-signed)

15. Database Schema & Seed Data

Skema database dibuat otomatis oleh Hibernate (ddl-auto=update) pada saat pertama kali aplikasi dijalankan. Skrip DDL berikut juga disertakan sebagai schema-seed.sql untuk inisialisasi manual pada lingkungan yang membatasi DDL runtime.

15.1. DDL - Tabel users

```
CREATE TABLE users (  
    id            BIGINT IDENTITY(1,1) PRIMARY KEY,  
    email         NVARCHAR(100) NOT NULL UNIQUE,  
    password      NVARCHAR(255) NOT NULL,  
    full_name     NVARCHAR(100) NOT NULL,  
    role          NVARCHAR(20)  NOT NULL CHECK (role IN ('ROLE_ADMIN', 'ROLE_USER')),  
    last_login    DATETIME2     NULL,  
    created_at    DATETIME2     NOT NULL DEFAULT SYSDATETIME(),  
    updated_at    DATETIME2     NULL  
);  
  
CREATE UNIQUE INDEX UX_users_email ON users(email);  
CREATE INDEX      IX_users_role   ON users(role);
```

15.2. DDL - Tabel tasks

```
CREATE TABLE tasks (  
    id            BIGINT IDENTITY(1,1) PRIMARY KEY,  
    title         NVARCHAR(200) NOT NULL,  
    description   NVARCHAR(MAX) NULL,  
    status        NVARCHAR(20)  NOT NULL DEFAULT 'TODO'  
                CHECK (status IN ('TODO', 'IN_PROGRESS', 'DONE')),  
    assigned_to   BIGINT        NULL,  
    created_at    DATETIME2     NOT NULL DEFAULT SYSDATETIME(),  
    updated_at    DATETIME2     NULL,  
    CONSTRAINT FK_tasks_users FOREIGN KEY (assigned_to)  
        REFERENCES users(id) ON DELETE SET NULL,  
    CONSTRAINT CK_tasks_title_length CHECK (LEN(title) >= 5)  
);  
  
CREATE INDEX IX_tasks_status      ON tasks(status);  
CREATE INDEX IX_tasks_assigned_to ON tasks(assigned_to);  
CREATE INDEX IX_tasks_title      ON tasks(title);
```

15.3. Seed Data - Pengguna Default

DataSeeder pada modul backend menjalankan upsert idempotent ketika tabel users kosong. Operator juga dapat menjalankan skrip berikut secara manual (password sudah di-hash BCrypt strength 10).

```
-- Password 'admin123' (BCrypt $2a$10$...)  
INSERT INTO users (email, password, full_name, role, created_at) VALUES  
('admin@taskmanager.com',
```

```
'$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhwy',  
'Administrator', 'ROLE_ADMIN', SYSDATETIME()),  
( 'user@taskmanager.com',  
'$2a$10$8K1p/a0dL1LXMIgoEDFrwOe6PpfTxWZ.2hZ9Q3Lj.0YqJk0p7e7VG',  
'Regular User', 'ROLE_USER', SYSDATETIME());
```

15.4. Seed Data - Contoh Tugas

```
INSERT INTO tasks (title, description, status, assigned_to, created_at) VALUES  
( 'Setup Development Environment',  
'Konfigurasi awal lingkungan pengembangan termasuk instalasi tools, dependencies, dan  
dokumentasi setup.',  
'DONE', 1, SYSDATETIME()),  
( 'Implement User Authentication',  
'Mengimplementasikan sistem autentikasi pengguna dengan JWT, registrasi, login, dan manajemen  
sesi.',  
'IN_PROGRESS', 2, SYSDATETIME()),  
( 'Design Database Schema',  
'Merancang dan mengimplementasikan skema database untuk sistem manajemen tugas dengan relasi  
yang optimal.',  
'TODO', 2, SYSDATETIME());
```

16. Installation & Deployment Guide

Bagian ini menyediakan langkah operasional untuk mereplikasi seluruh sistem dari source di lingkungan reviewer. Estimasi waktu total: 10–15 menit dengan koneksi internet stabil dan tools prasyarat telah terinstal.

16.1. Prasyarat Sistem

Komponen	Versi Minimum	Versi Direkomendasikan	Verifikasi
Java Development Kit (JDK)	17	OpenJDK 17 LTS / 21 LTS	java -version
Node.js & npm	16.x / 8.x	Node 20 LTS / npm 10.x	node -v && npm -v
Microsoft SQL Server	2017	2019 atau 2022 Express	sqlcmd -S localhost -Q "SELECT @@VERSION"
Angular CLI	16.0	16.x global atau via npx	ng version
Apache Maven	Opsional (mvnw tersedia)	3.9.x	mvn -v
Git	2.30+	Terbaru	git --version

16.2. Setup Database SQL Server

8. Pastikan SQL Server berjalan pada port 1433 dengan TCP/IP protocol enabled (cek via SQL Server Configuration Manager).
9. Pastikan SQL Server Authentication mode aktif (Mixed Mode) dan akun sa atau akun setara aktif.
10. Buat database melalui SSMS, Azure Data Studio, atau sqlcmd:

```
sqlcmd -S localhost -U sa -P 'YourPassword123!' -Q "
  IF DB_ID('task_management_db') IS NULL
    CREATE DATABASE task_management_db;
"
```

11. Tabel akan dibuat otomatis pada startup backend pertama via Hibernate ddl-auto=update; tidak perlu menjalankan DDL secara manual untuk evaluasi.

16.3. Setup Backend

12. Klon repository:

```
git clone https://github.com/Arfazr11/Sompo-Insurance-Assessment.git
cd Sompo-Insurance-Assessment/backend
```

13. Salin template environment dan sesuaikan kredensial:

```
cp .env.example .env
# Edit .env: ubah DB_PASSWORD jika berbeda dari default.
```

14. Jalankan aplikasi melalui Maven wrapper:

```
# Linux / macOS
./mvnw spring-boot:run

# Windows
mvnw.cmd spring-boot:run
```

15. Tunggu hingga log menampilkan baris berikut yang menandakan startup sukses:

```
Tomcat started on port 8080 (http) with context path ''
Started TaskManagementApplication in X.XXX seconds
[DataSeeder] Default users created successfully.
[DataSeeder] Sample tasks seeded successfully.
```

16. Verifikasi backend aktif melalui health-check sederhana:

```
curl http://localhost:8080/api/auth/users
# Akan mengembalikan 401 Unauthorized (benar - endpoint protected).
```

16.4. Setup Frontend

17. Buka terminal baru, masuk ke direktori frontend:

```
cd Sompo-Insurance-Assessment/frontend
```

18. Instalasi dependencies:

```
npm install
```

19. Verifikasi konfigurasi environment.ts mengarah ke backend yang benar (default: http://localhost:8080/api).

20. Jalankan development server Angular:

```
npm start
# atau
ng serve --open
```

21. Aplikasi akan terbuka otomatis di browser pada http://localhost:4200.

16.5. Build Produksi

16.5.1. Backend - Executable JAR

```
./mvnw clean package -DskipTests
# Output: backend/target/task-management-system-0.0.1-SNAPSHOT.jar

# Jalankan dengan environment variable
```

```
java -jar target/task-management-system-0.0.1-SNAPSHOT.jar \
  --spring.profiles.active=prod
```

16.5.2. Frontend - Static Bundle

```
ng build --configuration production
# Output: frontend/dist/task-management-system/
# Deploy folder hasil build ke web server statis (nginx, S3+CloudFront, dst.)
```

16.6. Catatan Deployment Produksi

- Gunakan reverse proxy (nginx / Traefik) di depan backend untuk TLS termination, gzip, dan rate-limiting.
- Pisahkan JWT_SECRET produksi dari nilai default; gunakan secret manager (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault).
- Set spring.jpa.hibernate.ddl-auto=validate atau none di produksi; gunakan Flyway/Liquibase untuk migrasi terkontrol.
- Aktifkan HTTPS-only cookies bila menyimpan token pada cookie, dan implementasikan refresh-token rotation untuk mempersingkat lifetime access token.
- Konfigurasi CORS_ALLOWED_ORIGINS dengan domain spesifik produksi - jangan pernah menggunakan wildcard.
- Aktifkan health-check endpoint via Spring Boot Actuator (/actuator/health) untuk integrasi dengan load balancer.

17. Testing & Quality Assurance

17.1. Static Code Analysis - SonarCloud

Repository terintegrasi dengan SonarCloud untuk analisis statis berkelanjutan pada setiap push ke branch utama. Pemindaian mencakup tujuh dimensi mutu: bug, vulnerability, security hotspot, code smell, duplications, coverage, dan complexity.

Dashboard publik: https://sonarcloud.io/summary/overall?id=Arfazrll_Sompo-Insurance-Assessment

17.1.1. Ringkasan Quality Gate

Dimensi	Target	Status
Bugs (Reliability)	0 Blocker / Critical	Passed
Vulnerabilities (Security)	0 Blocker / Critical	Passed
Security Hotspots	100% reviewed	Passed
Code Smells	Maintainability rating A	Passed
Duplications	< 3% pada code baru	Passed
Reliability Rating	A	Passed
Security Rating	A	Passed
Maintainability Rating	A	Passed

17.2. Manual Testing Checklist

17.2.1. Authentication & Authorization

- Login dengan kredensial valid (admin & user) mengembalikan JWT dan user info.
- Login dengan kredensial salah mengembalikan HTTP 401 dengan body { code: 'INVALID_CREDENTIALS' }.
- Registrasi dengan email duplikat mengembalikan HTTP 400 dengan pesan validasi yang jelas.
- Akses endpoint tanpa Authorization header mengembalikan HTTP 401 melalui JwtAuthenticationEntryPoint.
- Pengguna ROLE_USER yang mencoba mengakses POST/PUT/DELETE /api/tasks mendapatkan HTTP 403.
- Token kedaluwarsa (exp < now) memicu auto-logout di frontend dan redirect ke /login.

17.2.2. Task Management

- Admin dapat membuat, mengubah, dan menghapus tugas; aksi tersembunyi pada UI untuk pengguna biasa.
- Pengguna biasa hanya dapat mengubah status tugas yang ditugaskan kepadanya (validasi server-side).

- Pencarian by title bersifat case-insensitive dan mendukung partial match (LIKE %query%).
- Filter status menerima nilai TODO, IN_PROGRESS, DONE, atau ALL (tanpa filter).
- Validasi title minimum 5 karakter dijalankan di frontend (Validators.minLength) dan backend (@Size).
- Field assignedTo opsional; tugas tanpa assignee tetap dapat dibuat oleh Admin.

17.2.3. UI / UX

- Animated login illustration: mata karakter mengikuti pointer dengan smooth interpolation.
- MatSidenav responsif: mode 'side' di breakpoint desktop, mode 'over' di mobile.
- Snack-bar feedback muncul pada setiap operasi CRUD sukses / gagal.
- Confirm dialog wajib muncul sebelum penghapusan tugas.
- Form reset ke kondisi pristine setelah submit sukses.

17.3. API Testing - Postman Collection

Collection lengkap dengan environment variable, pre-request script (auto-login), dan assertion telah dipublikasikan untuk memudahkan reviewer melakukan smoke test.

URL Collection: <https://documenter.getpostman.com/view/39986471/2sBXwsK9br>

- **Auth folder:** Login (auto-set {{token}}), Register, Get All Users.
- **Tasks folder:** Get All (dengan query search/status), Get By ID, Create, Update, Update Status, Delete.
- **Environment:** baseUrl=http://localhost:8080/api, token (auto-populate), adminEmail, userEmail.

18. Default Credentials & Akses

Sistem menyertakan dua akun bawaan yang dibuat otomatis oleh DataSeeder pada startup pertama. Kredensial ini ditujukan eksklusif untuk evaluasi reviewer dan WAJIB diganti sebelum deployment produksi.

18.1. Akun Bawaan

Peran	Email	Password	Hak Akses
Administrator	admin@taskmanager.com	admin123	CRUD penuh pada semua tugas, melihat seluruh pengguna, assign ulang tugas.
Regular User	user@taskmanager.com	user1234	Melihat semua tugas, mengubah status tugas yang ditugaskan kepadanya.

18.2. Peringatan Keamanan

- Kredensial default ini terdokumentasi secara publik di repository - tidak boleh digunakan di lingkungan apa pun selain evaluasi internal.
- Sebelum deployment produksi: jalankan reset password manual via SQL atau endpoint admin, hapus DataSeeder dari komponen Spring (atau bungkus dalam `@Profile("!prod")`).
- Rotasi password dianjurkan minimum setiap 90 hari, dengan kebijakan kompleksitas terenforce di layer service.
- JWT_SECRET default juga harus diganti dengan nilai yang di-generate menggunakan crypto-secure RNG minimum 256 bit.

19. Compliance Matrix - Pemenuhan Brief Teknis

Matriks berikut memetakan setiap butir kebutuhan pada brief teknis dengan implementasi konkret pada source code. Seluruh kebutuhan wajib telah terpenuhi (status: Implemented), didukung lokasi file/kelas yang relevan untuk verifikasi cepat.

Kebutuhan dari Brief	Implementasi	Lokasi	Status
Backend menggunakan Spring Boot	Spring Boot 3.3.5 + Java 17 + Maven	backend/pom.xml	Implemented
Frontend menggunakan Angular	Angular 16 + TypeScript 5.x + Material	frontend/package.json	Implemented
Database SQL Server	MS SQL Server 2019+ via JDBC + JPA	application.properties	Implemented
JWT untuk autentikasi & otorisasi	JJWT 0.12.6, HS256, 24h expiration	util/JwtTokenProvider.java	Implemented
Manajemen peran (Admin & User)	RBAC enum Role + @PreAuthorize	model/User.java, controller/*	Implemented
CRUD tugas	5 endpoint REST (GET list, GET id, POST, PUT, DELETE)	controller/TaskController.java	Implemented
Fitur pencarian by title	JSQL searchByTitle case-insensitive LIKE	repository/TaskRepository.java	Implemented
Filter berdasarkan status	Query combinatorial title x status	service/TaskService.java	Implemented
Validasi input	Bean Validation (@NotBlank, @Size, @Email)	dto/* + model/*	Implemented
Pengelolaan password aman	BCrypt strength 10 + PasswordEncoder bean	config/SecurityConfig.java	Implemented
RESTful endpoint	Konvensi /api/{resource}, HTTP verb idiomatic, status code akurat	controller/*	Implemented
Penggunaan JPA / Hibernate	Spring Data JPA + HikariCP + SQLServerDialect	repository/*	Implemented
Route Guard di Angular	AuthGuard, GuestGuard, AdminGuard via CanActivate	guards/*.guard.ts	Implemented
Reactive Forms	FormBuilder + Validators di login, register, task form	components/*/*.component.ts	Implemented
HTTP Interceptor untuk JWT	JwtInterceptor inject Bearer + handle 401 auto-logout	interceptors/jwt.interceptor.ts	Implemented
Penanganan error terpusat	@RestControllerAdvice GlobalExceptionHandler	exception/GlobalExceptionHandler.java	Implemented

Kebutuhan dari Brief	Implementasi	Lokasi	Status
CORS configuration	CorsConfigurationSource + bean reference di security chain	config/SecurityConfig.java	Implemented
Stateless session	SessionCreationPolicy.STATELESS pada filter chain	config/SecurityConfig.java	Implemented
Penggunaan DTO	Request/Response DTO terpisah dari entity	dto/*Request.java, dto/*Response.java	Implemented
Dokumentasi API	Postman collection publik + bagian 10 dokumen ini	Postman + dokumen ini	Implemented
Quality gate eksternal	SonarCloud quality gate passed (Rating A semua dimensi)	sonarcloud.io	Implemented

20. Appendix - Tautan Resmi & Sumber Verifikasi

Seluruh artefak proyek tersedia untuk diakses publik oleh reviewer guna verifikasi independen atas klaim dalam dokumen ini.

Artefak	URL
Source Code Repository	https://github.com/Arfazrll/Sampo-Insurance-Assessment
API Documentation (Postman)	https://documenter.getpostman.com/view/39986471/2sBXwsK9br
Static Analysis Dashboard (SonarCloud)	https://sonarcloud.io/summary/overall?id=Arfazrll_Sampo-Insurance-Assessment

20.1. Struktur Penyerahan

- Source code lengkap pada repository GitHub publik.
- Dokumen teknis (file .docx ini) - disertakan pada repository pada path docs/ dan dilampirkan pada email submisi.
- Postman collection - siap import untuk smoke test seluruh endpoint.
- Tangkapan layar SonarCloud - embed badge tersedia pada README.md.

20.2. Kontak

Atribut	Nilai
Nama	Syahril Arfian Almazril
Email	azril4974@gmail.com
GitHub	https://github.com/Arfazrll
Portofolio	https://syahrilarfianalmazril.my.id
Posisi yang Dilamar	Java Developer

20.3. Pernyataan Penutup

Dokumen ini menyediakan rujukan teknis komprehensif atas implementasi Task Management System sebagai materi evaluasi Technical Assessment PT. Sampo Insurance Indonesia. Setiap klaim arsitektural, keamanan, dan fungsional dapat diverifikasi langsung melalui source code, Postman collection, dan dashboard SonarCloud yang tercantum di atas. Saya bersedia mempresentasikan, mendemonstrasikan secara live, dan menjawab pertanyaan lanjutan terkait setiap keputusan teknis yang diambil.

Hormat saya,

Syahril Arfian Almazril

21. Dokumentasi

The screenshots illustrate the 'Daftar Tugas' (Task List) feature for different users in the Task Management System.

Screenshot 1: Administrator View

The Administrator view shows a list of tasks assigned to them. The tasks are:

JUDUL	STATUS	DITUGASKAN KE	DIBUAT	AKSI
Setup project backend	Done	Administrator	10 Jun 2026	[Edit] [Delete]
Implementasi autentikasi JWT	In Progress	John Doe	10 Jun 2026	[Edit] [Delete]
Desain halaman dashboard	To Do	John Doe	10 Jun 2026	[Edit] [Delete]
Dokumentasi Teknikal	In Progress	Syahril Arfan A	10 Jun 2026	[Edit] [Delete]
Kumpulkan ke user	To Do	Syahril Arfan A	10 Jun 2026	[Edit] [Delete]

Screenshot 2: Syahril Arfan A View

The Syahril Arfan A view shows a filtered list of tasks assigned to them. The tasks are:

JUDUL	STATUS	DITUGASKAN KE	DIBUAT
Dokumentasi Teknikal	In Progress	Syahril Arfan A	10 Jun 2026
Kumpulkan ke user	To Do	Syahril Arfan A	10 Jun 2026

Screenshot 3: Administrator Dashboard View

The Administrator view of the Dashboard shows a summary of task progress:

- 5 Total Tugas
- 2 To Do
- 2 In Progress
- 1 Done

Progress Keseluruhan: 20% selesai

Tugas Terbaru:

- Setup project backend (Done) - Administrator
- Implementasi autentikasi JWT (In Progress) - John Doe
- Desain halaman dashboard (To Do) - John Doe
- Dokumentasi Teknikal (In Progress) - Syahril Arfan A
- Kumpulkan ke user (To Do) - Syahril Arfan A

